

CAPÍTULO 1

CONCEPTOS

ELEMENTALES



Introducción

La computación gráfica es una de las áreas de la informática visual, donde se utilizan las computadoras para generar imágenes o gráficos sintéticamente para integrar o cambiar la información visual y espacial del mundo real. La computación gráfica se aplica en la creación de: motores gráficos, motores de videojuegos, modelamiento de objetos 2D y 3D, efectos especiales para películas, procesamiento de imágenes, procesamiento de video, visión por computadora, entre otros (Shirley, P., Ashikhmin, M., 2009).

El padre de la computación gráfica es el científico de la computación *Ivan Sutherland*. A Ivan Sutherland le tomó varios años desarrollar el Sketchpad (ver Figura 1.1), que fue un programa informático creado en 1963 que permitía la manipulación directa de objetos gráficos, así como la interacción persona-computadora y fue el predecesor de los programas de diseño asistido por computadora o computer-aided design (CAD). Ivan Sutherland a lo largo de su vida recibió varios premios y reconocimientos como el Premio Turing de la Asociación de Maquinaria Computacional, el cual es considerado el premio más importante en las ciencias de la computación, en 1988. En el 2012 recibió el Premio Kioto de Tecnología Avanzada por “logros pioneros en el desarrollo de la computación gráfica e interfaces interactivas”. Además, este científico de la computación es miembro de Academia Nacional de Ingeniería de Estados Unidos, así como de la Academia Nacional de Ciencias de Estados Unidos (Hughes, J.F., et al., 2013).



Figura 1.1. El Dr. Sutherland operando el Sketchpad (Webster, R., Clark, A., 2015).

La computación gráfica cubre conceptos y aspectos relacionados a las matemáticas, cálculo y geometría aplicada a la computación. En muchas aplicaciones se necesitan manejar puntos, líneas, figuras geométricas y polígonos para representar objetos y modelos en 2D y 3D.

Objetivos

- Revisar los principales conceptos en torno a algoritmos para la solución de problemas de gráficos por computadora, enfocados a figuras geométricas.
- Entender cómo trabajan los algoritmos y las gráficos por computadora con el lenguaje C# en la construcción de programas, enfocados a figuras geométricas.
- Aprender a crear algoritmos de gráficos por computadora en Pseudocódigo y en el lenguaje C#, enfocados a figuras geométricas.
- Diseñar y crear programas con gráficos por computadora en el lenguaje C# y programación orientada a objetos con interfaces gráficas de usuario, enfocados a figuras geométricas.
- Resolver casos de estudio y practiques con algoritmos y programación orientada a objetos aplicados a los fundamentos de la Computación Gráfica.

1.1. Píxeles, Coordenadas y Figuras Geométricas

La forma más conveniente de especificar un segmento de línea en la pantalla de una computadora es proporcionando las coordenadas de sus dos puntos finales. En matemáticas, las coordenadas son números reales, pero las rutinas primitivas de dibujo de líneas pueden requerir que sean enteros. Este es el caso, por ejemplo, en el lenguaje C#, que se utilizará a lo largo de este libro. El componente o subsistema GDI (Graphics Device Interface) trabaja junto con el núcleo y la API de Windows (Steeb, W.H., Hardy, A., 2008). Esta interfaz de programación de aplicaciones se encarga del control gráfico de los dispositivos de salida, como los monitores o las impresoras. Las tareas más comunes de GDI son:

- El dibujo de líneas, curvas, polígonos;
- El relleno de cuadros, círculos, polígonos, etcétera;
- Se encarga del renderizado de fuentes y textos;
- Se encarga del manejo de paletas de colores.

1.1.1. Graficación de Líneas

El componente GDI proporciona la clase `Graphics` que contiene el método `DrawLine`, que se lo utiliza de la siguiente manera para dibujar el segmento de línea que conecta A y B (ver Figura 1.2):

```
g.DrawLine(xA, yA, xB, yB);
```

El contexto gráfico `g` delante del método normalmente se proporciona como un parámetro del método de dibujo en el programa, y los cuatro argumentos de `DrawLine` son enteros, que van desde cero hasta algún valor máximo (Ammeraal, L., Zhang, K., 2017). La llamada anterior a `DrawLine` produce exactamente la misma salida en la pantalla que esta, donde se conecta B y A:

```
g.DrawLine(xB, yB, xA, yA);
```



Figura 1.2. Dibujo de una línea a través de dos puntos.

1.1.2. Graficación de Rectángulos

Para graficar un rectángulo se utiliza el contexto `g` con la función `DrawRectangle` (ver Figura 1.3), y se pueden utilizar cinco parámetros como son la pluma de dibujo, las coordenadas iniciales a partir de donde se grafica el rectángulo, el valor del ancho y el largo del rectángulo (Ammeraal, L., Zhang, K., 2017).

```
g.DrawRectangle(Pen, 0, 0, Width, Height);
```



Figura 1.3. Dibujo de un rectángulo.

1.1.3. Graficación de Cuadrados

Para graficar un cuadrado se utiliza el contexto g con la función DrawRectangle (ver Figura 1.4), y se pueden utilizar cinco parámetros como son la pluma de dibujo, las coordenadas iniciales a partir de donde se grafica el cuadrado (Lengyel, E., 2011). En este caso el valor del ancho y del largo son iguales, por lo que se coloca en ambos parámetros el valor del lado del cuadrado, puesto que el cuadrado es un caso especial del rectángulo.

```
g.DrawRectangle(Pen, 0, 0, Side, Side);
```

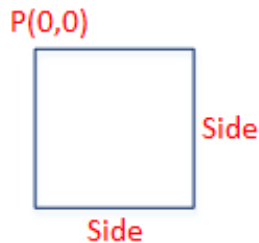


Figura 1.4. Dibujo de un cuadrado a partir de un rectángulo.

1.1.4. Graficación de Elipses

Para graficar una elipse se utiliza el contexto g con la función DrawEllipse (ver Figura 1.5), y se pueden utilizar cinco parámetros como son la pluma de dibujo, las coordenadas iniciales a partir de donde se grafica la elipse, el valor del ancho y el largo de la elipse (Ammeraal, L., Zhang, K., 2017).

```
g.DrawEllipse(Pen, 0, 0, Width, Height);
```

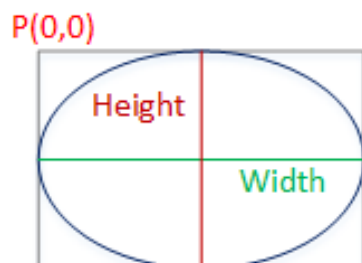


Figura 1.5. Dibujo de una elipse.

1.1.5. Graficación de Círculos

Para graficar un círculo se utiliza el contexto g con la función DrawEllipse (ver Figura 1.6), y se pueden utilizar cinco parámetros como son la pluma de dibujo, las coordenadas iniciales a partir de donde se grafica el círculo (Lengyel, E., 2011). En este caso el valor del ancho y del largo de la elipse son iguales y equivalentes al diámetro del círculo, por lo que se coloca en ambos parámetros el valor del diámetro del círculo, puesto que el círculo es un caso especial de la elipse.

```
g.DrawEllipse(Pen, 0, 0, Diameter, Diameter);
```

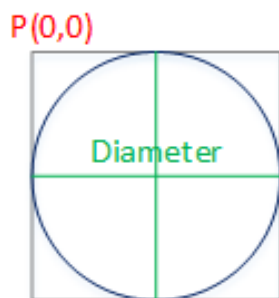


Figura 1.6. Dibujo de un círculo a partir de una elipse.

1.1.6. Ejes Coordenados por Pantalla

El eje de coordenadas por pantalla del mundo de la computación gráfica es un eje en el cual se conserva el eje positivo de las 'x' hacia la derecha tal y como el del mundo real, pero el eje positivo de las 'y' es hacia abajo, debido a que el estándar de fabricación de los monitores y pantallas se basa en el movimiento de los electrones desde arriba hacia abajo y desde la izquierda hacia la derecha aplicando los principios de la energía potencial eléctrica, donde el origen de coordenadas $O(0,0)$ está en la esquina superior izquierda (ver Figura 1.7).

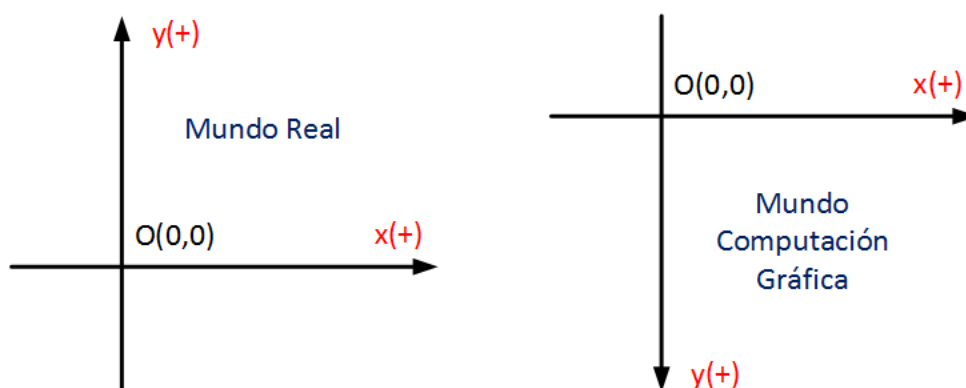


Figura 1.7. Eje de coordenadas del mundo real y del mundo de la Computación Gráfica.

1.2. Pinceles y Relleno de Figuras en GDI+

La interfaz de programación de aplicaciones GDI+, proporciona una serie de clases de pincel (brush) para rellenar los interiores de figuras geométricas cerradas como: círculos, elipses, rectángulos, cuadrados, triángulos y polígonos. Una figura cerrada consta de un entorno (outline) y un interior, donde el contorno se dibuja con un bolígrafo

(pen) y el interior se rellena con un pincel (brush). Para el relleno de figuras cerradas se pueden utilizar las clases: SolidBrush, HatchBrush, TextureBrush, LinearGradientBrush y PathGradientBrush, donde todas estas clases se derivan de la clase Brush (Steeb, W.H., Hardy, A., 2008). La Figura 1.8 muestra un rectángulo relleno con un pincel sólido y una elipse rellena con un pincel de sombreado.

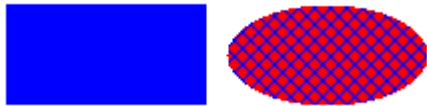


Figura 1.8. Rectángulo relleno y elipse rellena.

1.2.1. Pinceles Sólidos (Solid Brushes)

Para rellenar una figura geométrica cerrada se debe utilizar un objeto de la clase Graphics y un pincel de la clase Brush. El objeto de la clase Graphics proporciona métodos como: FillRectangle, FillEllipse y FillPolygon (Steeb, W.H., Hardy, A., 2008). El objeto de la clase Brush almacena los atributos del relleno como el color. El objeto pincel (brush) se envía como uno de los argumentos a los parámetros de la función: FillRectangle, FillEllipse y FillPolygon. El siguiente código de ejemplo muestra cómo rellenar un rectángulo con un color azul sólido:

```
SolidBrush mySolidBrush = new SolidBrush(Color.Blue);  
myGraphics.FillRectangle(mySolidBrush, 0, 0, 70, 20);
```

1.2.2. Pinceles de Sombreado o de Escotilla (Hatch Brushes)

Para rellenar una figura geométrica con un pincel de sombreado, se debe especificar un color de primer plano, un color de fondo y un estilo de sombreado, donde el color de primer plano es el color de la trama (hatching). La interfaz de programación de aplicaciones GDI+, proporciona más de 50 estilos de sombreado (Steeb, W.H., Hardy, A., 2008). Un código de ejemplo se muestra a continuación con un estilo de sombreado Vertical:

```
HatchBrush myHatchBrush =  
    new HatchBrush(HatchStyle.Vertical, Color.Blue, Color.Green);
```

La Figura 1.9 muestra tres ejemplos de pinceles de sombreado: Horizontal, Diagonal hacia adelante (ForwardDiagonal) y Cruz (Cross).

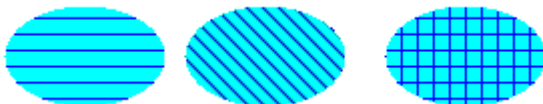


Figura 1.9. Ejemplos de pinceles de sombreado.

1.2.3. Pinceles de Textura (Texture Brushes)

Para rellenar una figura geométrica con una textura almacenada en un mapa de bits (bitmap), se puede utilizar un pincel de textura (Steeb, W.H., Hardy, A., 2008). Así, por

ejemplo, si se tiene una imagen almacenada en un archivo en el disco duro llamado MyTexture.bmp, se puede asignar a una elipse un relleno con esta imagen. El siguiente código de ejemplo muestra lo expuesto:

```
Image myImage = Image.FromFile("MyTexture.bmp");
TextureBrush myTextureBrush = new TextureBrush(myImage);
myGraphics.FillEllipse(myTextureBrush, 0, 0, 100, 50);
```

La Figura 1.10 muestra la elipse rellena con una imagen de un archivo de mapa de bits.



Figura 1.10. Ejemplo de relleno de una elipse con una imagen.

1.2.4. Pinceles de Degradado (Gradient Brushes)

Para rellenar una figura geométrica con un pincel de degradado, se debe especificar un primer color de plano que cambia gradualmente a un segundo color de plano a medida que se mueve a lo largo de la figura geométrica de manera horizontal, vertical o diagonalmente (Steeb, W.H., Hardy, A., 2008). La interfaz de programación de aplicaciones GDI+, proporciona dos tipos de pinceles de degradado que son el lineal y el de ruta. El siguiente código de ejemplo muestra cómo rellenar una elipse con un pincel de degradado horizontal que cambia de azul a verde a medida que se mueve desde el borde izquierdo de la elipse al borde derecho.

```
LinearGradientBrush myLinearGradientBrush = new LinearGradientBrush(
    myRectangle,
    Color.Blue,
    Color.Green,
    LinearGradientMode.Horizontal);
myGraphics.FillEllipse(myLinearGradientBrush, myRectangle);
```

La Figura 1.11 muestra la elipse rellena con un pincel de degradado.



Figura 1.11. Ejemplo de relleno de una elipse con un pincel de degradado.

1.2.5. Relleno de un Polígono (Fill Polygon)

Para rellenar un polígono se debe cumplir con los siguientes pasos: 1) Crear un objeto de tipo pincel (brush) con un color de relleno; 2) Crear un arreglo de puntos (Point/PointF) para definir un polígono; 3) Rellenar el área poligonal correspondiente al polígono pintado en la pantalla, utilizando la función **FillPolygon** (Steeb, W.H., Hardy, A., 2008). El siguiente código de ejemplo está diseñado para su uso con Windows Forms

y requiere un objeto 'e' de tipo *PaintEventArgs*, que es un parámetro del controlador del evento *Paint*.

```
public void FillPolygonPointF(PaintEventArgs e)
{
    // Crear un pincel sólido de relleno.
    SolidBrush blueBrush = new SolidBrush(Color.Blue);

    // Crear siete puntos que definen un polígono irregular.
    PointF point1 = new PointF(50.0F, 50.0F);
    PointF point2 = new PointF(100.0F, 25.0F);
    PointF point3 = new PointF(200.0F, 5.0F);
    PointF point4 = new PointF(250.0F, 50.0F);
    PointF point5 = new PointF(300.0F, 100.0F);
    PointF point6 = new PointF(350.0F, 200.0F);
    PointF point7 = new PointF(250.0F, 250.0F);

    // Crear un arreglo de objetos de tipo PointF para siete puntos.
    PointF[] curvePoints = {point1, point2, point3, point4, point5, point6, point7};

    // Rellenar el área poligonal en la pantalla.
    e.Graphics.FillPolygon(blueBrush, curvePoints);
}
```


1.3. Casos de Estudio Resueltos

Caso de Estudio 1.1: Graficar un Rectángulo y Encontrar el Perímetro y su Área.

A) Problema

Escribir un programa para graficar un rectángulo, calcular e imprimir el perímetro y su área. (Ver Figura 1.1.1)

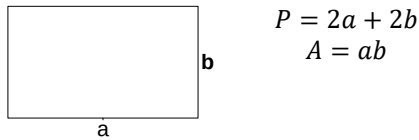


Figura 1.1.1. El rectángulo y sus fórmulas.

B) Análisis

Claramente, se puede ver que las entradas del problema son el **ancho** y el **largo** del rectángulo. Hay tres salidas requeridas: el **perímetro**, el **área** del rectángulo y el **gráfico** de la figura geométrica. Partiendo de un conocimiento básico de geometría, se sabe que hay una relación entre el ancho y el largo del rectángulo con el perímetro y con el área de la figura geométrica. Las fórmulas requeridas se incluyen en el requerimiento de los datos.

B.1) Requerimiento de los Datos

Entradas del Problema

width	/* ancho del rectángulo */
height	/* largo del rectángulo */

Salidas del Problema

perimeter	/* perímetro del rectángulo */
area	/* área del rectángulo */
graph	/* objeto que activa el modo gráfico */

Auxiliares del Problema

pen	/* Pluma de dibujo */
-----	-----------------------

Constantes del Problema

SF = 20	/* factor de escalamiento */
---------	------------------------------

Fórmulas Relevantes

$P = 2a + 2b$	(1)	/* Fórmula del perímetro de un rectángulo */
$A = ab$	(2)	/* Fórmula del área de un rectángulo */

B.2) Diagrama de Entrada-Salida

En la Figura 1.2.2 se muestra el diagrama de Entrada-Salida del programa donde se identifican y se diagraman las entradas, salidas y auxiliares del problema como son: a)

Entradas: las variables 'width' y 'height'; b) Salidas: la variable 'perimetro', la variable 'area' y el objeto 'graph'; c) Auxiliares: el objeto 'pen'; d) Constantes: el factor de escalamiento 'SF'.

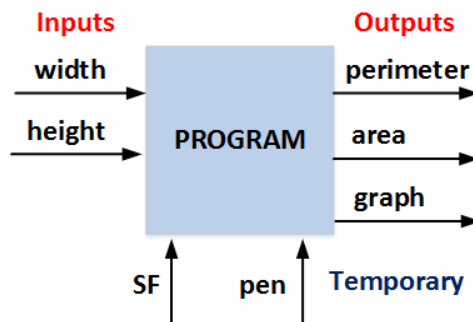


Figura 1.1.2. Diagrama de Entrada-Salida del Programa.

B.3) Diagrama de Clases

El diagrama de clases de esta aplicación se muestra en la Figura 1.1.3 con notación UML. En este diagrama se puede ver que la aplicación está compuesta por seis clases, donde la clase **Program** se compone por la clase **frmRectangle** que resulta ser una clase derivada de la clase **Form**. La clase **frmRectangle** se compone de un objeto de tipo **CRectangle**.

La clase **CRectangle** se compone de un objeto de tipo **Graphics** llamado **mGraph**, que maneja el modo gráfico de la aplicación y de un objeto de tipo **Pen** llamado **mPen**, para dibujar sobre un lienzo representado por un **pictureBox** cuyo identificador es **picCanvas**. Además, se tiene una constante llamada **SF** (scale factor) de tipo float que permite realizar un acercamiento (Zoom-In) y un alejamiento (Zoom-Out) sobre la figura geométrica. Los otros atributos que se tienen en la clase **CRectangle** son el ancho, el largo, el perímetro y el área representados por los datos miembros **mWidth**, **mHeight**, **mPerimeter** y **mArea** que son de tipo **float**.

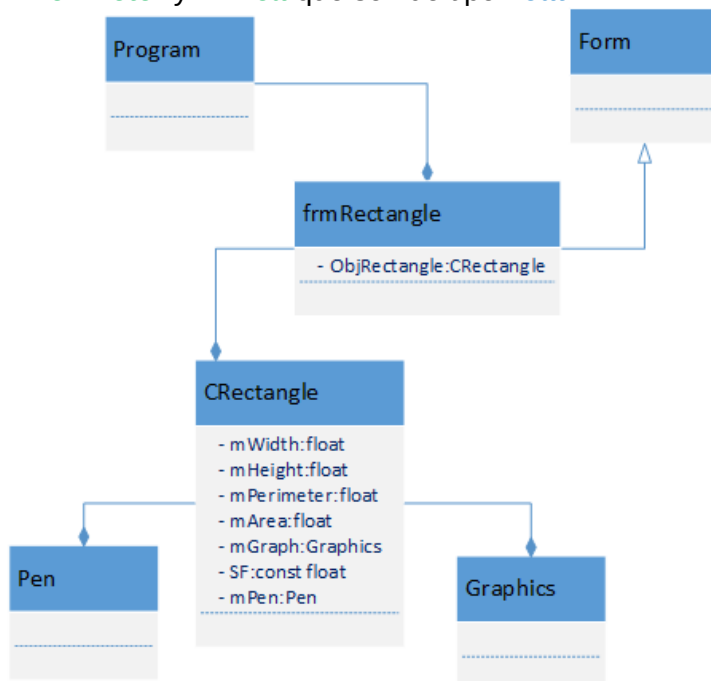


Figura 1.1.3. Diagrama de clases del Caso de Estudio 1.1.

C) Diseño

Una vez que se conoce las entradas y las salidas del problema, se deben listar los pasos necesarios para resolver el problema, es decir, el algoritmo.

C.1) Algoritmos

3.1.1. Algoritmo de la Función *ReadData()*

1. Leer el ancho del rectángulo.
2. Leer el largo del rectángulo.

3.1.2. Algoritmo de la Función *PerimeterRectangle()*

1. Calcular el perímetro.
 - 1.1. Asignar el producto de dos veces el ancho por dos veces el largo a la variable 'perimeter'.

3.1.3. Algoritmo de la Función *AreaRectangle()*

1. Calcular el área.
 - 1.1. Asignar el producto del ancho por el largo a la variable 'area'.

3.1.4. Algoritmo de la Función *PrintData()*

1. Imprimir el valor de la variable 'perimeter' y de la variable 'area'.

3.1.5. Algoritmo de la Función *PlotShape()*

1. Graficar un rectángulo.
 - 1.1. Utilizar el objeto 'Graph' con la función DrawRectangle para graficar con la pluma 'Pen' un rectángulo.

D) Implementación

Para implementar la solución, se deben escribir los algoritmos como una **Aplicación para Windows** con **C# .NET** que contenga toda la información necesaria para completar la solución de dicho problema y pueda la aplicación ser ejecutada desde **Visual Studio .NET**.

Para comenzar la construcción de esta aplicación, empiece por ejecutar el Entorno de Desarrollo Integrado (IDE – Integrated Development Environment) de Visual Studio 2019 y seleccione **Crear un proyecto**, como se muestra en la Figura 1.1.4.

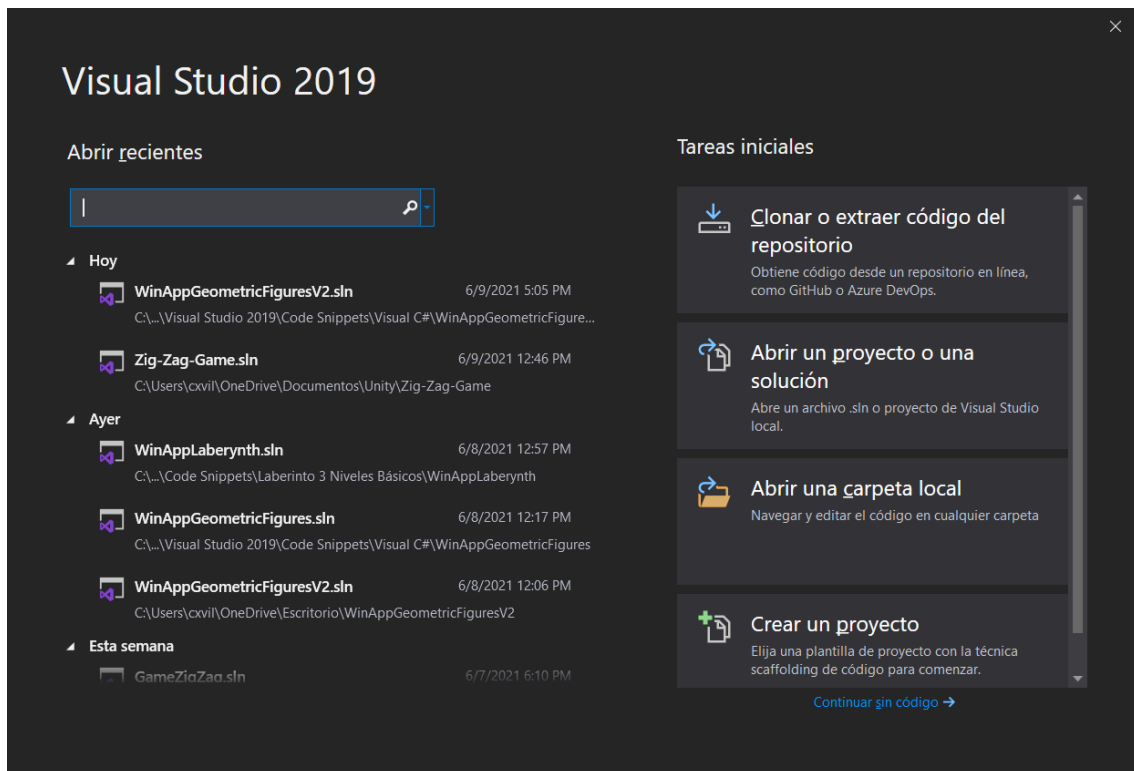


Figura 1.1.4. Creación de un proyecto en C# .NET.

A continuación, en el cuadro de diálogo de Crear un proyecto se debe seleccionar la Plantilla (Template) correspondiente a la **Aplicación de Windows Forms** (.NET Framework) con el **lenguaje C#** y luego presione el botón de Siguiente, como se muestra en la Figura 1.1.5.

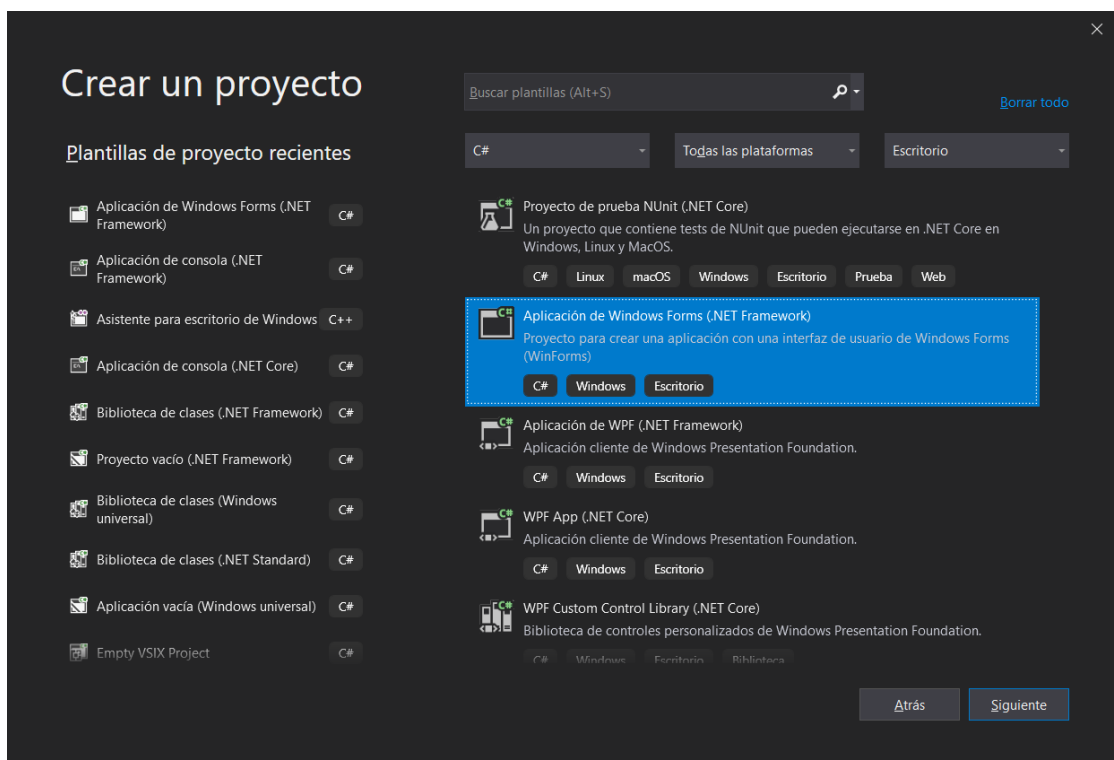


Figura 1.1.5. Creación de una Aplicación de Windows Forms.

Computación Gráfica en C#

Luego, aparece un cuadro de diálogo de Configure su nuevo proyecto, donde se debe modificar el nombre de la aplicación con **WinAppRectangle** y se debe seleccionar una localidad para guardar en el disco duro dicho proyecto como, por ejemplo: C:\Users\cxvill\source\repos. (Ver Figura 1.1.6).

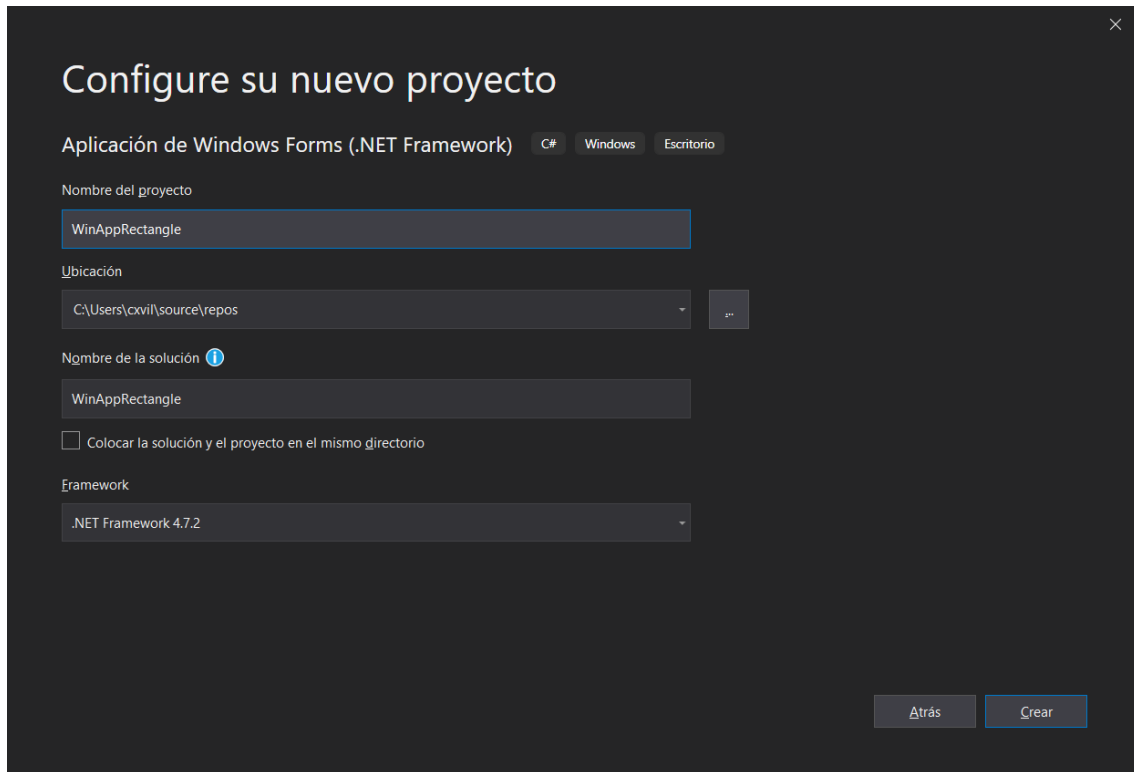


Figura 1.1.6. Configuración de un nuevo proyecto.

Finalmente se crea la Aplicación de Windows Forms cuyo nombre es **WinAppRectangle** en el IDE de **C# .NET**. Como se puede ver en la Figura 1.1.7, el ambiente de trabajo de la aplicación consta de seis partes: 1) Barra de Menús; 2) Barra de Tareas; 3) Caja de Herramientas; 4) Explorador de Soluciones; 5) Ventana de Propiedades; 6) Formulario (Form1). (Ver Figura 1.1.7).

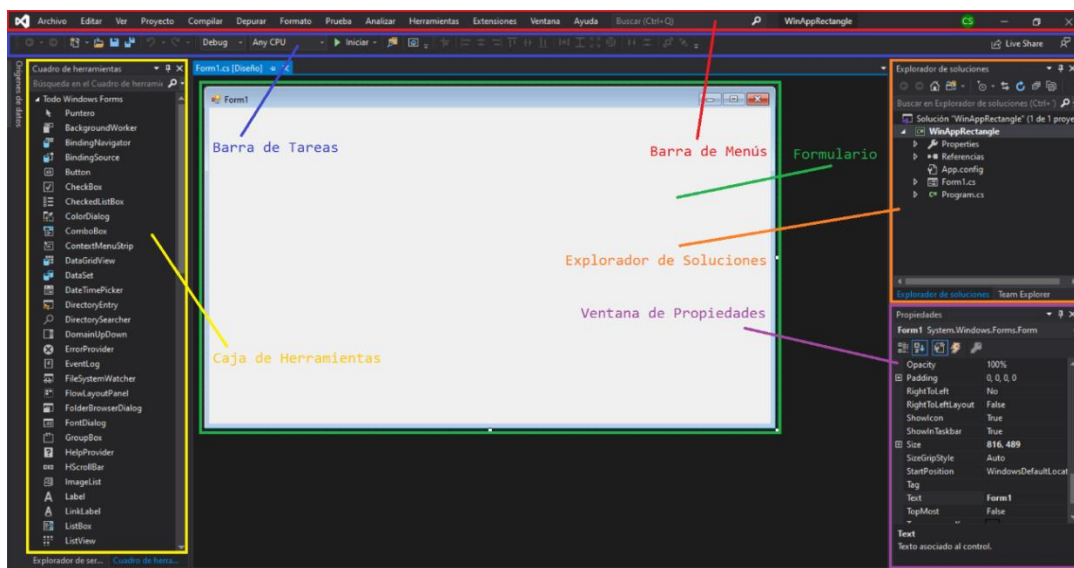


Figura 1.1.7. IDE de C# .NET.

Ahora que ya se ha guardado el proyecto, acceda al **Explorador de Soluciones** (Solution Explorer) y modifique el nombre del archivo del formulario a **frmRectangle.cs**, cuyo nombre por defecto es **Form1.cs** y que consta dentro de la Aplicación de Windows Forms con el nombre de **WinAppRectangle**, como se muestra en la Figura 1.1.8. Además, cree la clase llamada **CRectangle** con la cual va a trabajar el formulario frmRectangle, aplicando el concepto de Composición y Agregación.

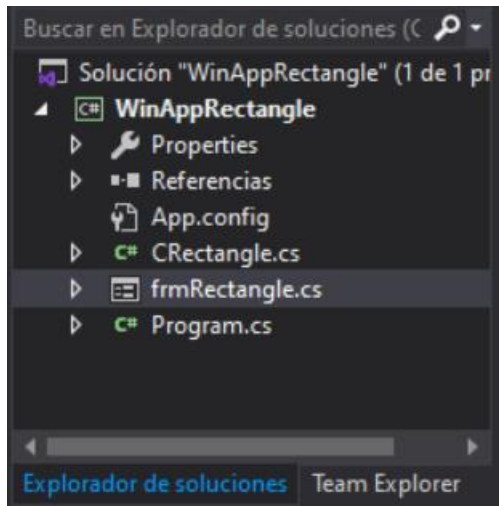


Figura 1.1.8. Modificación del nombre del Formulario en la Aplicación.

Se necesita añadir al **Formulario** (Form) tres **Cajas de Grupo** (GroupBoxes), cuatro **Etiquetas** (Labels), cuatro **Cajas de Texto** (TextBoxes), tres **Botones** (Buttons), y una **Caja de Imagen** (PictureBox), dispuestos de la siguiente manera: a) La primera **Caja de Grupo** (GroupBox), contiene dos **Etiquetas** (Label) y dos **Cajas de Texto** (TextBox); b) La segunda **Caja de Grupo** (GroupBox), contiene tres **Botones** (Button); c) la tercera **Caja de Grupo** (Group Box), contiene con dos **Etiquetas** (Labels) y dos **Cajas de Texto** (TextBoxes); la cuarta **Caja de Grupo** (GroupBox), contiene un **Caja de Imagen** (PictureBox). Disponga los controles de tal manera que se asemeje a la Figura 1.1.9 que se muestra a continuación.

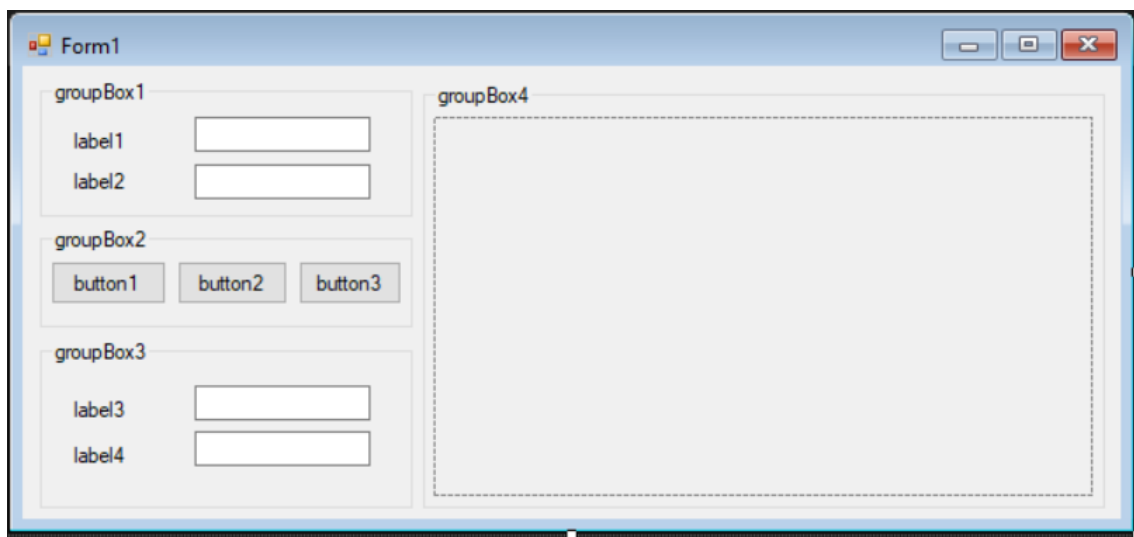


Figura 1.1.9. Disposición de los Controles dentro del Formulario.

Utilice la Tabla 1.1.1, para modificar las propiedades restantes de cada objeto.

Tabla 1.1.1. Objetos y sus Propiedades.

Objeto	Propiedad	Valor
Form	Name Text StartPosition	frmRectangle Rectángulo CenterScreen
GroupBox	Name Text Font	grbInputs Entradas MS Sans Serif, Bold, 8
Label	Name AutoSize Text	lblWidth True Ancho (a):
Label	Name AutoSize Text	lblHeight True Largo (b):
TextBox	Name Text	txtWidth Blank
TextBox	Name Text	txtHeight Blank
GroupBox	Name Text Font	grbProcess Proceso MS Sans Serif, Bold, 8
Button	Name Text	btnCalculate Calcular
Button	Name Text	btnReset Resetear
Button	Name Text	btnExit Salir
GroupBox	Name Text Font	grbOutputs Salidas MS Sans Serif, Bold, 8
Label	Name AutoSize Text	lblPerimeter True Perímetro:
Label	Name AutoSize Text	lblArea True Área:
TextBox	Name Text Enabled Locked	txtPerimeter Blank False True

TextBox	Name Text Enabled Locked	txtArea Blank False True
GroupBox	Name Text Font	grbCanvas Gráfico MS Sans Serif, Bold, 8
PictureBox	Name	picCanvas

Una vez modificadas las propiedades del formulario y de sus controles queda configurado de la siguiente manera, como se puede ver en la Figura 1.1.10.

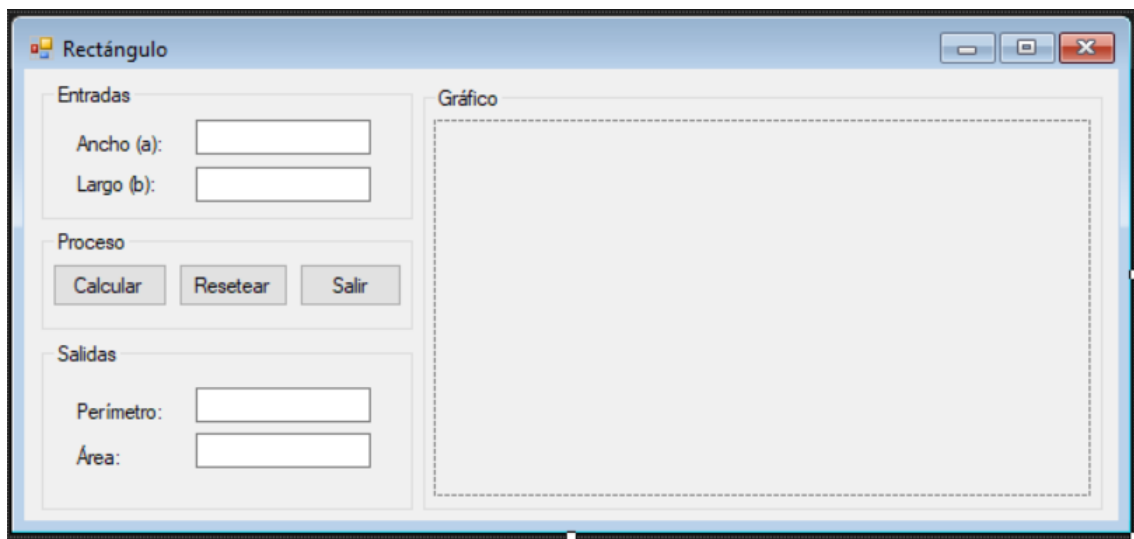


Figura 1.1.10. El Formulario y sus controles modificados sus propiedades.

Esta aplicación aceptará el ingreso de valores flotantes o enteros en dos **Cajas de Texto** (TextBox) y desplegará los resultados en otras dos **Cajas de Texto** (TextBoxes) y el gráfico de la figura se desplegará en una **Caja de Imagen** (PictureBox).

La clase **CRectangle** tiene los siguientes datos miembro: a) **mWidth**, **mHeight**, que representan al ancho y al largo del rectángulo que son de tipo **float**; b) **mPerimeter**, **mArea**, que representan al perímetro y al área del rectángulo que son también de tipo **float**; c) un objeto **mGraph** de tipo Graphics; d) un esferográfico de dibujo llamado **mPen** de tipo Pen; e) una constante de tipo **float** llamada **SF** definida como un factor de escalamiento (scale factor).

Esta clase también posee un grupo de métodos de acuerdo a las acciones que puede hacer y que básicamente serán funciones para poder representar y operar un rectángulo en el plano, entre las más importantes están las siguientes:

- a) **Constructor sin argumentos.** Este método inicializa los primeros cuatro datos miembros de la clase **CRectangle** con el valor de cero.

```
// Constructor sin parámetros.
public CRectangle()
```

```
{
    mWidth = 0.0f; mHeight = 0.0f;
    mPerimeter = 0.0f; mArea = 0.0f;
}
```

- b) **Función miembro ReadData.** Esta función permite leer los valores del ancho y del largo del rectángulo ingresados en cajas de texto. En este caso es necesario realizar un casting o conversión de tipo de datos de String a float utilizando la función `float.Parse()` y además se utiliza la función `try-catch` para el manejo y control de excepciones, en caso de que el usuario en lugar de ingresar números ingrese letras por error.

```
// Función que lee los datos de entrada del rectángulo.
public void ReadData(TextBox txtWidth, TextBox txtHeight)
{
    try
    {
        mWidth = float.Parse(txtWidth.Text);
        mHeight = float.Parse(txtHeight.Text);
    }
    catch
    {
        MessageBox.Show("Ingreso no válido...", "Mensaje de error");
    }
}
```

- a) **Función miembro Perimeter.** Esta función permite calcular el perímetro de un rectángulo, para lo cual se utiliza una fórmula matemática para poder realizar de esta manera los respectivos cálculos.

```
// Función que calcula el perímetro del rectángulo.
public void PerimeterRectangle()
{
    mPerimeter = 2 * mWidth + 2 * mHeight;
}
```

- b) **Función miembro Area.** Esta función permite calcular el área de un rectángulo, para lo cual se utiliza una fórmula matemática para poder realizar de esta manera los respectivos cálculos.

```
// Función que calcula el área del rectángulo.
public void AreaRectangle()
{
    mArea = mWidth * mHeight;
}
```

- c) **Función miembro PrintData.** Esta función permite imprimir en cajas de texto los valores del perímetro y del área del rectángulo calculados, para lo cual es necesario realizar un casting o conversión de tipo `float` a String, utilizando la función `ToString()`.

```
// Función que imprime el perímetro y el área del rectángulo.
public void PrintData(TextBox txtPerimeter, TextBox txtArea)
{
    txtPerimeter.Text = mPerimeter.ToString();
    txtArea.Text = mArea.ToString();
}
```

- d) **Función miembro InitializeData.** Esta función permite inicializar los datos miembro y los controles del formulario con valores por defecto, como en el caso de las **Cajas de Texto** se inicializan con el valor de [BLANK] utilizando los símbolos de comillas dobles (""). Para inicializar la **Caja de Imagen** llamada **picCanvas** se utiliza la función **Refresh** que significa refrescar o borrar.

```
// Función que inicializa los datos y controles del rectángulo.
public void InitializeData(TextBox txtWidth, TextBox txtHeight,
                          TextBox txtPerimeter, TextBox txtArea,
                          PictureBox picCanvas)
{
    mWidth = 0.0f; mHeight = 0.0f;
    mPerimeter = 0.0f; mArea = 0.0f;

    txtWidth.Text = ""; txtHeight.Text = "";
    txtPerimeter.Text = ""; txtArea.Text = "";
    // Mantiene el cursor titilando en una caja de texto.
    txtWidth.Focus();
    picCanvas.Refresh();
}
```

- e) **Función miembro PlotShape.** Esta función permite graficar un rectángulo, para lo cual en el objeto **mGraph** se asigna la funcionalidad de crear gráficos de la **Caja de Imagen** llamada **picCanvas**. Luego se crea un bolígrafo de color azul con una punta de ancho 3 conocida como **Brush**. Finalmente, se dibuja un rectángulo en las coordenadas (0,0) y con los valores del **ancho** (width) y del **largo** (height) multiplicados por un factor de escalamiento, para que la figura no sea muy pequeña.

```
// Función que grafica un rectángulo.
public void PlotShape(PictureBox picCanvas)
{
    mGraph = picCanvas.CreateGraphics();
    mPen = new Pen(Color.Blue, 3);
    // Graficar un rectángulo.
    mGraph.DrawRectangle(mPen, 0, 0, mWidth * SF, mHeight * SF);
}
```

- f) **Función miembro CloseForm.** Esta función permite cerrar un formulario utilizando la función **Close()**, que es llamada por el objeto **ObjForm** el cual es un parámetro de la función que recibe un objeto de tipo **Form**.

```
// Función que cierra un formulario.
public void CloseForm(Form ObjForm)
{
    ObjForm.Close();
}
```

Tips de Programación

Se utiliza la función **float.Parse()** para convertir un tipo de dato carácter o cadena a tipo flotante.

Se utiliza la función **ToString()** para convertir un tipo de dato numérico a tipo carácter o cadena.

La implementación de la clase **CRectangle**, se presenta a continuación en la Tabla 1.1.2, que contiene datos miembro y funciones miembro que realizan diferentes operaciones.

Tabla 1.1.2. Código de la clase **CRectangle** del programa.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Drawing;
using System.Windows.Forms;

namespace WinAppRectangle
{
    class CRectangle
    {
        // Datos Miembro (Atributos).

        // Ancho del rectángulo.
        private float mWidth;
        // Largo del rectángulo.
        private float mHeight;
        // Perímetro del rectángulo.
        private float mPerimeter;
        // Área del rectángulo.
        private float mArea;
        // Objeto que activa el modo gráfico.
        private Graphics mGraph;
        // Constante scale factor (Zoom In/Zoom Out).
        private const float SF = 20;
        // Objeto bolígrafo que dibuja o escribe en un lienzo (canvas).
        private Pen mPen;

        // Funciones Miembro (Métodos).

        // Constructor sin parámetros.
        public CRectangle()
        {
            mWidth = 0.0f; mHeight = 0.0f;
            mPerimeter = 0.0f; mArea = 0.0f;
        }

        // Función que lee los datos de entrada del rectángulo.
        public void ReadData(TextBox txtWidth, TextBox txtHeight)
        {
            try
            {
                mWidth = float.Parse(txtWidth.Text);
                mHeight = float.Parse(txtHeight.Text);
            }
            catch
            {
                MessageBox.Show("Ingreso no válido...",
                                "Mensaje de error");
            }
        }

        // Función que calcula el perímetro del rectángulo.
        public void PerimeterRectangle()
        {
            mPerimeter = 2 * mWidth + 2 * mHeight;
        }

        // Función que calcula el área del rectángulo.
        public void AreaRectangle()
```

```
{
    mArea = mWidth * mHeight;
}

// Función que imprime el perímetro y el área del rectángulo.
public void PrintData(TextBox txtPerimeter, TextBox txtArea)
{
    txtPerimeter.Text = mPerimeter.ToString();
    txtArea.Text = mArea.ToString();
}

// Función que inicializa los datos y controles del rectángulo.
public void InitializeData(TextBox txtWidth, TextBox txtHeight,
    TextBox txtPerimeter, TextBox txtArea,
    PictureBox picCanvas)
{
    mWidth = 0.0f; mHeight = 0.0f;
    mPerimeter = 0.0f; mArea = 0.0f;

    txtWidth.Text = ""; txtHeight.Text = "";
    txtPerimeter.Text = ""; txtArea.Text = "";
    // Mantiene el cursor titilando en una caja de texto.
    txtWidth.Focus();
    picCanvas.Refresh();
}

// Función que grafica un rectángulo.
public void PlotShape(PictureBox picCanvas)
{
    mGraph = picCanvas.CreateGraphics();
    mPen = new Pen(Color.Blue, 3);
    // Graficar un rectángulo.
    mGraph.DrawRectangle(mPen, 0, 0, mWidth * SF, mHeight * SF);
}

// Función que cierra un formulario.
public void CloseForm(Form ObjForm)
{
    ObjForm.Close();
}
}
```

La implementación de la clase *frmRectangle*, se presenta a continuación en la Tabla 1.1.3, que es la clase que representa a la interfaz gráfica de usuario de la figura geométrica conocida como **Rectángulo**.

Tabla 1.1.3: Código de la clase *frmRectangle* del programa.

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace WinAppRectangle
{
    public partial class frmRectangle : Form
```

```
{
    // Definición de un objeto de tipo CRectangle.
    private CRectangle ObjRectangle = new CRectangle();

    public frmRectangle()
    {
        InitializeComponent();
    }

    private void frmRectangle_Load(object sender, EventArgs e)
    {
        // Inicialización de los datos y controles.
        // Llamada a la función InitializeData.
        ObjRectangle.InitializeData(txtWidth, txtHeight,
                                   txtPerimeter, txtArea,
                                   picCanvas);
    }

    private void btnCalculate_Click(object sender, EventArgs e)
    {
        // Lectura de datos.
        // Llamada a la función ReadData.
        ObjRectangle.ReadData(txtWidth, txtHeight);
        // Cálculo del perímetro de un rectángulo.
        // Llamada a la función PerimeterRectangle.
        ObjRectangle.PerimeterRectangle();
        // Cálculo del área de un rectángulo.
        // Llamada a la función AreaRectangle.
        ObjRectangle.AreaRectangle();
        // Impresión de datos.
        // Llamada a la función PrintData.
        ObjRectangle.PrintData(txtPerimeter, txtArea);
        // Graficación de un rectángulo.
        // Llamada a la función PlotShape.
        ObjRectangle.PlotShape(picCanvas);
    }

    private void btnReset_Click(object sender, EventArgs e)
    {
        // Inicialización de los datos y controles.
        // Llamada a la función InitializeData.
        ObjRectangle.InitializeData(txtWidth, txtHeight,
                                   txtPerimeter, txtArea,
                                   picCanvas);
    }

    private void btnExit_Click(object sender, EventArgs e)
    {
        // Cierre de un formulario.
        // Llamada a la función CloseForm.
        ObjRectangle.CloseForm(this);
    }
}
```

La clase **frmRectangle** tiene como datos miembro un objeto de tipo **CRectangle** llamado **ObjRectangle**, el cual es el encargado de acceder a las funciones públicas de la clase **CRectangle** para realizar diferentes cálculos y operaciones.

En el evento **Load** del formulario, cuyo procedimiento se llama **frmRectangle_Load()**, se utiliza el objeto **ObjRectangle**, para llamar a la función pública **InitializeData()**, que

se encarga de inicializar los datos y controles del formulario. En el evento **Click** del botón **btnCalculate**, cuyo procedimiento se llama **btnCalculate_Click()**, se utiliza el objeto **ObjRectangle**, para llamar a las funciones públicas: **ReadData()**, **PerimeterRectangle()**, **AreaRectangle()**, **PrintData()** y **PlotShape()**. En el evento **Click** del botón **btnReset**, cuyo procedimiento se llama **btnReset_Click()**, se utiliza el objeto **ObjRectangle**, para llamar a la función pública **InitializeData()**. En el evento **Click** del botón **btnClose**, cuyo procedimiento se llama **btnClose_Click()**, se utiliza el objeto **ObjRectangle**, para llamar a la función pública **CloseForm()**, que permite cerrar el formulario.

E) Pruebas

Para correr la aplicación basta con presionar la **tecla F5** o el **botón Start** de ejecución de la barra de menús de **Visual Studio .NET**. La Figura 1.1.11 muestra un ejemplo de la corrida de este programa, pero para saber que los resultados del problema son correctos, se debe examinar los resultados del programa cuidadosamente para asegurarnos que tengan sentido. El ejemplo mostrado de la salida del programa proporciona una buena prueba de la solución porque es relativamente fácil calcular el área y el perímetro manualmente o utilizando una calculadora. Para un valor de ancho de 8 y de largo de 5 se puede observar la gráfica de la figura geométrica en la **Caja de Imagen** llamada **picCanvas**. El perímetro deberá ser dos veces el ancho más dos veces el largo, que también es un valor fácil de calcular manualmente o utilizando una calculadora. El área deberá ser igual al producto del ancho por el largo, que también es un valor fácil de calcular manualmente o utilizando una calculadora. Para verificar que el programa funciona adecuadamente, se deberá ingresar unos pocos valores de prueba correspondientes al ancho y al largo de un rectángulo. No se necesita probar más que con unos pocos casos de prueba para verificar que este programa está correcto.

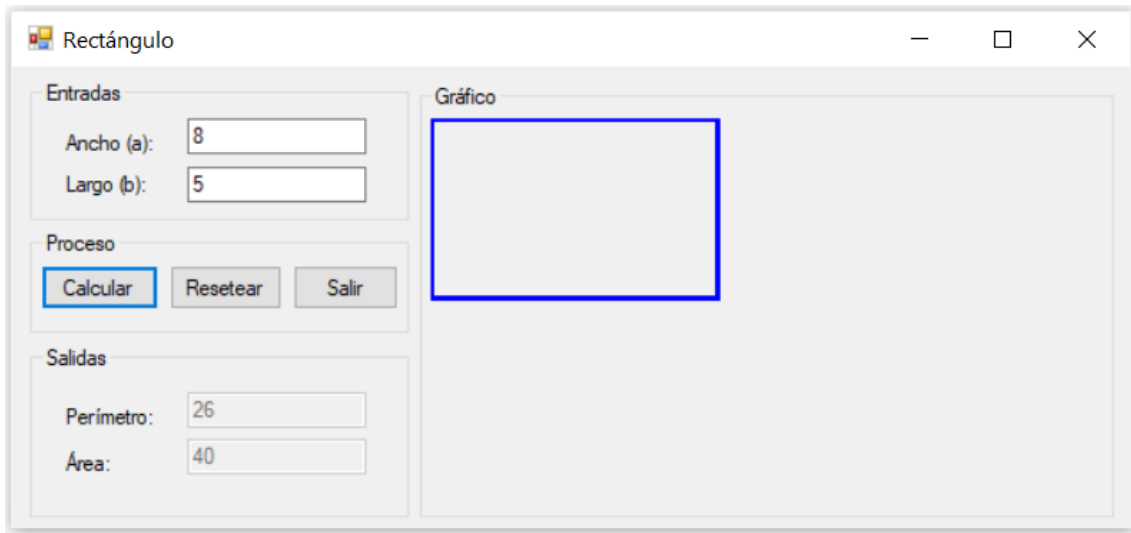


Figura 1.1.11. Ejemplo de la Corrida del Programa.

Caso de Estudio 1.2: Graficar un Cuadrado y Encontrar el Perímetro y su Área.

A) Problema

Escribir un programa para graficar un cuadrado, calcular e imprimir el perímetro y su área. (Ver Figura 1.3.1)

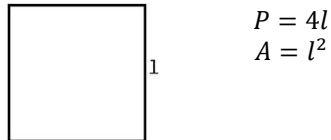


Figura 1.2.1. El cuadrado y sus fórmulas.

B) Análisis

Claramente, se puede ver que la entrada del problema es el lado del cuadrado. Hay tres salidas requeridas: el perímetro, el área del círculo y el gráfico de la figura geométrica. Partiendo de un conocimiento básico de geometría, se sabe que hay una relación entre el lado del cuadrado con el perímetro y con el área de la figura geométrica. Las fórmulas requeridas se incluyen en el requerimiento de los datos.

B.1) Requerimiento de los Datos

Entradas del Problema

side /* lado del cuadrado */

Salidas del Problema

perimeter /* perímetro del cuadrado */
area /* área del cuadrado */
graph /* objeto que activa el modo gráfico */

Auxiliares del Problema

pen /* Pluma de dibujo */

Constantes del Problema

SF = 20 /* factor de escalamiento */

Fórmulas Relevantes

$P = 4l$ (1) /* Fórmula del perímetro de un cuadrado */
 $A = l^2$ (2) /* Fórmula del área de un cuadrado */

B.2) Diagrama de Entrada-Salida

En la Figura 1.2.2 se muestra el Diagrama de Entrada-Salida del programa donde se identifican y se diagraman las entradas, salidas y auxiliares del problema como son: a) Entradas: la variable 'side'; b) Salidas: la variable 'perimetro', la variable 'area' y el objeto 'graph'; c) Auxiliares: el objeto 'pen'; d) Constantes: el factor de escalamiento 'SF'.

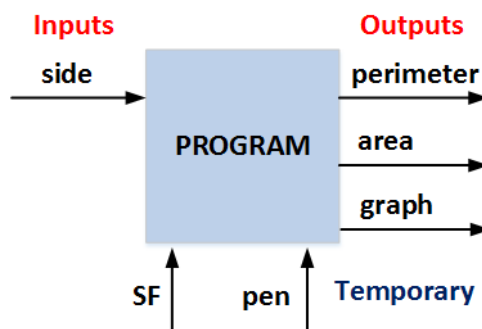


Figura 1.2.2. Diagrama de Entrada-Salida del Programa.

B.3) Diagrama de Clases

El diagrama de clases de esta aplicación se muestra en la Figura 1.2.3 con notación UML. En este diagrama se puede ver que la aplicación está compuesta por seis clases, donde la clase **Program** se compone por la clase **frmSquare** que resulta ser una clase derivada de la clase **Form**. La clase **frmSquare** se compone de un objeto de tipo **CSquare**.

La clase **CSquare** se compone de un objeto de tipo **Graphics** llamado **mGraph**, que maneja el modo gráfico de la aplicación y de un objeto de tipo **Pen** llamado **mPen**, para dibujar sobre un lienzo representado por un **pictureBox** cuyo identificador es **picCanvas**. Además, se tiene una constante llamada **SF** (scale factor) de tipo float que permite realizar un acercamiento (Zoom-In) y un alejamiento (Zoom-Out) sobre la figura geométrica. Los otros atributos que se tienen en la clase **CSquare** son el lado, el perímetro y el área representados por los datos miembros **mSide**, **mPerimeter** y **mArea** que son de tipo **float**.

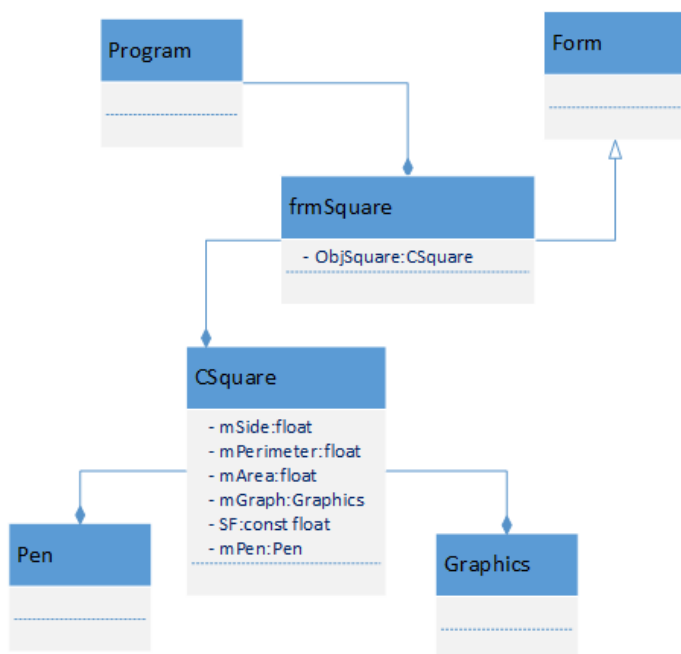


Figura 1.2.3. Diagrama de clases del Caso de Estudio 1.2.

C) Diseño

Una vez que se conoce las entradas y las salidas del problema, se deben listar los pasos necesarios para resolver el problema, es decir, el algoritmo.

C.1) Algoritmos

3.1.1. Algoritmo de la Función *ReadData()*

1. Leer el lado del cuadrado.

3.1.2. Algoritmo de la Función *PerimeterSquare()*

1. Calcular el perímetro.
 - 1.1. Asignar el producto de cuatro veces el lado a la variable 'perimeter'.

3.1.3. Algoritmo de la Función *AreaSquare()*

1. Calcular el área.
 - 1.1. Asignar el cuadrado del lado a la variable 'area'.

3.1.4. Algoritmo de la Función *PrintData()*

1. Imprimir el valor de la variable 'perimeter' y de la variable 'area'.

3.1.5. Algoritmo de la Función *PlotShape()*

1. Graficar un cuadrado.
 - 1.1. Utilizar el objeto 'Graph' con la función DrawRectangle para graficar con la pluma 'Pen' un cuadrado.

D) Implementación

Para implementar la solución, se deben escribir los algoritmos como una **Aplicación para Windows** con **C# .NET** que contenga toda la información necesaria para completar la solución de dicho problema y pueda la aplicación ser ejecutada desde **Visual Studio .NET**.

Para comenzar la construcción de esta aplicación, se debe seleccionar la opción de Crear un proyecto y la Plantilla (template) correspondiente a la **Aplicación de Windows Forms** con el **lenguaje C#**. En el cuadro de diálogo configure su proyecto con el nombre de **WinAppSquare** y luego guarde el mismo en el disco duro.

Ahora que ya se ha guardado el proyecto, acceda al **Explorador de Soluciones** (Solution Explorer) y modifique el nombre del archivo del formulario a **frmSquare.cs**, cuyo nombre por defecto es **Form1.cs** y que consta dentro de la **Aplicación de Windows Forms** con el nombre de **WinAppSquare**, como se muestra en la Figura 1.2.4. Además, cree la clase llamada **CSquare** con la cual va a trabajar el formulario **frmSquare**, aplicando el concepto de Composición y Agregación.

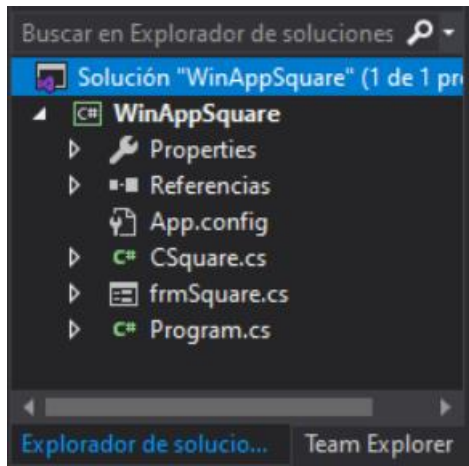


Figura 1.2.4. Solución WinAppSquare.

Se necesita añadir al **Formulario** (Form) tres **Cajas de Grupo** (GroupBoxes), cuatro **Etiquetas** (Labels), tres **Cajas de Texto** (TextBoxes), tres **Botones** (Buttons), y una **Caja de Imagen** (PictureBox), dispuestos de la siguiente manera: a) La primera **Caja de Grupo** (GroupBox), contiene una **Etiqueta** (Label) y una **Caja de Texto** (TextBox); b) La segunda **Caja de Grupo** (GroupBox), contiene tres **Botones** (Button); c) la tercera **Caja de Grupo** (Group Box), contiene con dos **Etiquetas** (Labels) y dos **Cajas de Texto** (TextBoxes); la cuarta **Caja de Grupo** (GroupBox), contiene un **Caja de Imagen** (PictureBox).

Utilice la Tabla 1.2.1, para modificar las propiedades restantes de cada objeto.

Tabla 1.2.1. Objetos y sus Propiedades.

Objeto	Propiedad	Valor
Form	Name Text StartPosition	frmCircle Círculo CenterScreen
GroupBox	Name Text Font	grbInputs Entradas MS Sans Serif, Bold, 8
Label	Name AutoSize Text	lblSide True Radio:
TextBox	Name Text	txtSide Blank
GroupBox	Name Text	grbProcess Blank
Button	Name Text	btnCalculate Calcular
Button	Name Text	btnReset Resetear

Computación Gráfica en C#

Button	Name Text	btnExit Salir
GroupBox	Name Text Font	grbOutputs Salidas MS Sans Serif, Bold, 8
Label	Name AutoSize Text	lblPerimeter True Perímetro:
Label	Name AutoSize Text	lblArea True Área:
TextBox	Name Text Enabled Locked	txtPerimeter Blank False True
TextBox	Name Text Enabled Locked	txtArea Blank False True
GroupBox	Name Text Font	grbCanvas Gráfico MS Sans Serif, Bold, 8
PictureBox	Name	picCanvas

Una vez modificadas las propiedades del formulario y de sus controles queda configurado de la siguiente manera, como se puede ver en la Figura 1.2.5.

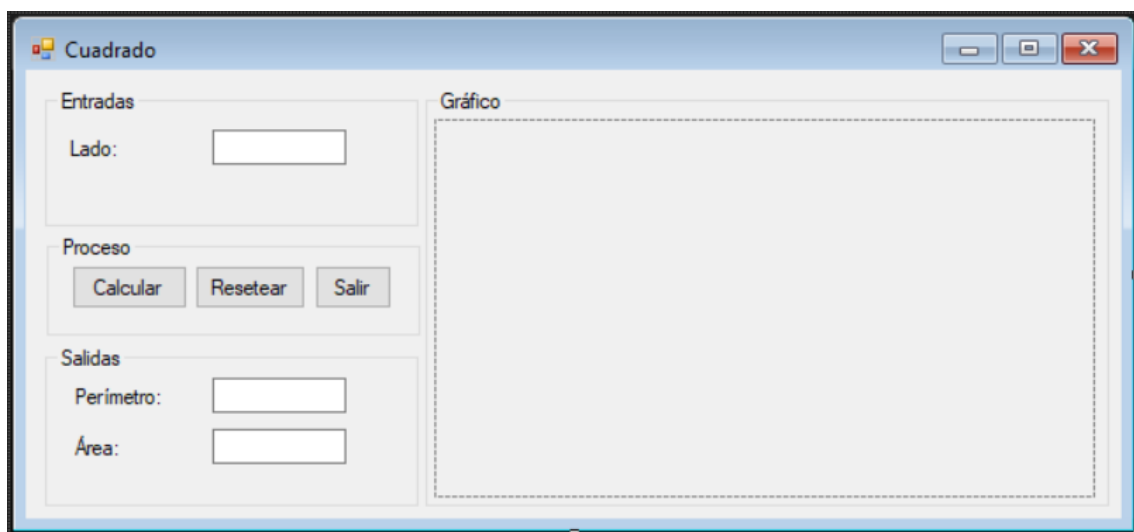


Figura 1.2.5. El Formulario y sus controles modificados sus propiedades.

Esta aplicación aceptará el ingreso de un valor flotante o entero en una **Caja de Texto** (TextBox) y desplegará los resultados en otras dos **Cajas de Texto** (TextBoxes) y el gráfico de la figura se desplegará en una **Caja de Imagen** (PictureBox).

La clase **CSquare** tiene los siguientes datos miembro: a) **mSide**, que representa al lado del cuadrado que es de tipo **float**; b) **mPerimeter**, **mArea**, que representan al perímetro y al área del cuadrado que son también de tipo **float**; c) un objeto **mGraph** de tipo Graphics; d) un esferográfico de dibujo llamado **mPen** de tipo Pen; e) una constante de tipo **float** llamada **SF** definida como un factor de escalamiento (scale factor).

Esta clase también posee un grupo de métodos de acuerdo a las acciones que puede hacer y que básicamente serán funciones para poder representar y operar un cuadrado en el plano, entre las más importantes están las siguientes:

- a) **Constructor sin argumentos.** Este método inicializa los primeros tres datos miembros de la clase **CSquare** con el valor de cero.

```
// Constructor sin parámetros.
public CSquare()
{
    mSide = 0.0f; mPerimeter = 0.0f; mArea = 0.0f;
}
```

- b) **Función miembro ReadData.** Esta función permite leer el valor del lado del cuadrado ingresado en una caja de texto. En este caso es necesario realizar un casting o conversión de tipo de datos de String a **float** utilizando la función **float.Parse()** y además se utiliza la función **try-catch** para el manejo y control de excepciones, en caso de que el usuario en lugar de ingresar números ingrese letras por error.

```
// Función que lee el lado del cuadrado.
public void ReadData(TextBox txtSide)
{
    try
    {
        mSide = float.Parse(txtSide.Text);
    }
    catch
    {
        MessageBox.Show("Ingreso no válido...", "Mensaje de error");
    }
}
```

- c) **Función miembro Perimeter.** Esta función permite calcular el perímetro de un cuadrado, para lo cual se utiliza una fórmula matemática para poder realizar de esta manera los respectivos cálculos.

```
// Función que calcula el perímetro del cuadrado.
public void PerimeterSquare()
{
    mPerimeter = 4 * mSide;
}
```

- d) **Función miembro Area.** Esta función permite calcular el área de un cuadrado, para lo cual se utiliza una fórmula matemática para poder realizar de esta manera los respectivos cálculos.

```
// Función que calcula el área del cuadrado.
```

```
public void AreaSquare()
{
    mArea = (float)Math.Pow(mSide, 2);
}
```

- e) **Función miembro PrintData.** Esta función permite imprimir en cajas de texto los valores del perímetro y del área del cuadrado calculados, para lo cual es necesario realizar un casting o conversión de tipo *float* a String, utilizando la función **ToString()**.

```
// Función que imprime el perímetro y el área del cuadrado.
public void PrintData(TextBox txtPerimeter, TextBox txtArea)
{
    txtPerimeter.Text = mPerimeter.ToString();
    txtArea.Text = mArea.ToString();
}
```

- f) **Función miembro InitializeData.** Esta función permite inicializar los datos miembro y los controles del formulario con valores por defecto, como en el caso de las **Cajas de Texto** se inicializan con el valor de [BLANK] utilizando los símbolos de comillas dobles (""). Para inicializar la **Caja de Imagen** llamada **picCanvas** se utiliza la función **Refresh** que significa refrescar o borrar.

```
// Función que inicializa los datos y controles del cuadrado.
public void InitializeData(TextBox txtSide, TextBox txtPerimeter,
                          TextBox txtArea, PictureBox picCanvas)
{
    mSide = 0.0f; mPerimeter = 0.0f; mArea = 0.0f;

    txtSide.Text = ""; txtPerimeter.Text = ""; txtArea.Text = "";
    // Mantiene el cursor titilando en una caja de texto.
    txtSide.Focus();
    picCanvas.Refresh();
}
```

- g) **Función miembro PlotShape.** Esta función permite graficar un círculo, para lo cual en el objeto **mGraph** se asigna la funcionalidad de crear gráficos de la **Caja de Imagen** llamada **picCanvas**. Luego se crea una pluma de color azul con una punta de ancho 3 conocida como **Brush**. Finalmente, se dibuja un cuadrado en las coordenadas (0,0) y con los valores del lado asignados al **ancho** (width) y al **largo** (height) del rectángulo multiplicados por un factor de escalamiento, para que la figura no sea muy pequeña.

```
// Función que grafica un cuadrado.
public void PlotShape(PictureBox picCanvas)
{
    mGraph = picCanvas.CreateGraphics();
    mPen = new Pen(Color.Blue, 3);
    // Graficar un cuadrado en función de un rectángulo.
    mGraph.DrawRectangle(mPen, 0, 0, mSide * SF, mSide * SF);
}
```

- h) **Función miembro CloseForm.** Esta función permite cerrar un formulario utilizando la función **Close()**, que es llamada por el objeto **ObjForm** el cual es un parámetro de la función que recibe un objeto de tipo **Form**.

```
// Función que cierra un formulario.
public void CloseForm(Form ObjForm)
{
}
```

```
        ObjForm.Close();  
    }
```

Tips de Programación

Se utiliza la función **float.Parse()** para convertir un tipo de dato carácter o cadena a tipo flotante.

Se utiliza la función **ToString()** para convertir un tipo de dato numérico a tipo carácter o cadena.

La implementación de la clase **CSquare**, se presenta a continuación en la Tabla 1.2.2, que contiene datos miembro y funciones miembro que realizan diferentes operaciones.

Tabla 1.2.2. Código de la clase **CSquare** del programa.

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
using System.Drawing;  
using System.Windows.Forms;  
  
namespace WinAppSquare  
{  
    class CSquare  
    {  
        // Datos Miembro (Atributos).  
  
        // Lado del cuadrado.  
        private float mSide;  
        // Perímetro del cuadrado.  
        private float mPerimeter;  
        // Área del cuadrado.  
        private float mArea;  
        // Objeto que activa el modo gráfico.  
        private Graphics mGraph;  
        // Constante scale factor (Zoom In/Zoom Out).  
        private const float SF = 20;  
        // Objeto bolígrafo que dibuja o escribe en un lienzo (canvas).  
        private Pen mPen;  
  
        // Funciones Miembro (Métodos).  
  
        // Constructor sin parámetros.  
        public CSquare()  
        {  
            mSide = 0.0f; mPerimeter = 0.0f; mArea = 0.0f;  
        }  
  
        // Función que lee el lado del cuadrado.  
        public void ReadData(TextBox txtSide)  
        {  
            try  
            {  
                mSide = float.Parse(txtSide.Text);  
            }  
            catch  
            {  
                MessageBox.Show("Ingreso no válido...", "Mensaje de error");  
            }  
        }  
    }  
}
```



```
// Función que calcula el perímetro del cuadrado.
public void PerimeterSquare()
{
    mPerimeter = 4 * mSide;
}

// Función que calcula el área del cuadrado.
public void AreaSquare()
{
    mArea = (float)Math.Pow(mSide, 2);
}

// Función que imprime el perímetro y el área del cuadrado.
public void PrintData(TextBox txtPerimeter, TextBox txtArea)
{
    txtPerimeter.Text = mPerimeter.ToString();
    txtArea.Text = mArea.ToString();
}

// Función que inicializa los datos y controles del cuadrado.
public void InitializeData(TextBox txtSide, TextBox txtPerimeter,
    TextBox txtArea, PictureBox picCanvas)
{
    mSide = 0.0f; mPerimeter = 0.0f; mArea = 0.0f;

    txtSide.Text = ""; txtPerimeter.Text = ""; txtArea.Text = "";
    // Mantiene el cursor titilando en una caja de texto.
    txtSide.Focus();
    picCanvas.Refresh();
}

// Función que grafica un cuadrado.
public void PlotShape(PictureBox picCanvas)
{
    mGraph = picCanvas.CreateGraphics();
    mPen = new Pen(Color.Blue, 3);
    // Graficar un cuadrado en función de un rectángulo.
    mGraph.DrawRectangle(mPen, 0, 0, mSide * SF, mSide * SF);
}

// Función que cierra un formulario.
public void CloseForm(Form ObjForm)
{
    ObjForm.Close();
}
}
```

La implementación de la clase *frmSquare*, se presenta a continuación en la Tabla 1.3.3, que es la clase que representa a la interfaz gráfica de usuario de la figura geométrica conocida como **Cuadrado**.

Tabla 1.3.3: Código de la clase *frmSquare* del programa.

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
```

```
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace WinAppSquare
{
    public partial class frmSquare : Form
    {
        // Definición de un objeto de tipo CSquare.
        CSquare ObjSquare = new CSquare();

        public frmSquare()
        {
            InitializeComponent();
        }

        private void frmSquare_Load(object sender, EventArgs e)
        {
            // Inicialización de los datos y controles.
            // Llamada a la función InitializeData.
            ObjSquare.InitializeData(txtSide, txtPerimeter,
                                    txtArea, picCanvas);
        }

        private void btnCalculate_Click(object sender, EventArgs e)
        {
            // Lectura de datos.
            // Llamada a la función ReadData.
            ObjSquare.ReadData(txtSide);
            // Cálculo del perímetro de un cuadrado.
            // Llamada a la función PerimeterSquare.
            ObjSquare.PerimeterSquare();
            // Cálculo del área de un cuadrado.
            // Llamada a la función AreaSquare.
            ObjSquare.AreaSquare();
            // Impresión de datos.
            // Llamada a la función PrintData.
            ObjSquare.PrintData(txtPerimeter, txtArea);
            // Graficación de un cuadrado.
            // Llamada a la función PlotShape.
            ObjSquare.PlotShape(picCanvas);
        }

        private void btnReset_Click(object sender, EventArgs e)
        {
            // Inicialización de los datos y controles.
            // Llamada a la función InitializeData.
            ObjSquare.InitializeData(txtSide, txtPerimeter,
                                    txtArea, picCanvas);
        }

        private void btnExit_Click(object sender, EventArgs e)
        {
            // Cierre de un formulario.
            // Llamada a la función CloseForm.
            ObjSquare.CloseForm(this);
        }
    }
}
```

La clase **frmSquare** tiene como datos miembro un objeto de tipo **CSquare** llamado **ObjSquare**, el cual es el encargado de acceder a las funciones públicas de la clase **CSquare** para realizar diferentes cálculos y operaciones.

En el evento **Load** del formulario, cuyo procedimiento se llama **frmCircle_Load()**, se utiliza el objeto **ObjSquare**, para llamar a la función pública **InitializeData()**, que se encarga de inicializar los datos y controles del formulario. En el evento **Click** del botón **btnCalculate**, cuyo procedimiento se llama **btnCalculate_Click()**, se utiliza el objeto **ObjSquare**, para llamar a las funciones públicas: **ReadData()**, **PerimeterSquare()**, **AreaSquare()**, **PrintData()** y **PlotShape()**. En el evento **Click** del botón **btnReset**, cuyo procedimiento se llama **btnReset_Click()**, se utiliza el objeto **ObjSquare**, para llamar a la función pública **InitializeData()**. En el evento **Click** del botón **btnClose**, cuyo procedimiento se llama **btnClose_Click()**, se utiliza el objeto **ObjSquare**, para llamar a la función pública **CloseForm()**, que permite cerrar el formulario.

E) Pruebas

Para correr la aplicación basta con presionar la **tecla F5** o el **botón Start** de ejecución de la barra de menús de **Visual Studio .NET**. La Figura 1.1.11 muestra un ejemplo de la corrida de este programa, pero para saber que los resultados del problema son correctos, se debe examinar los resultados del programa cuidadosamente para asegurarnos que tengan sentido. El ejemplo mostrado de la salida del programa proporciona una buena prueba de la solución porque es relativamente fácil calcular el área y el perímetro manualmente o utilizando una calculadora. Para un valor de un lado igual a 7 se puede observar la gráfica de la figura geométrica en la **Caja de Imagen** llamada **picCanvas**. El perímetro deberá ser cuatro veces el lado, que también es un valor fácil de calcular manualmente o utilizando una calculadora. El área deberá ser igual al lado elevado al cuadrado, que también es un valor fácil de calcular manualmente o utilizando una calculadora. Para verificar que el programa funciona adecuadamente, se deberá ingresar unos pocos valores de prueba correspondientes a los lados de algunos cuadrados. No se necesita probar más que con unos pocos casos de prueba para verificar que este programa está correcto.

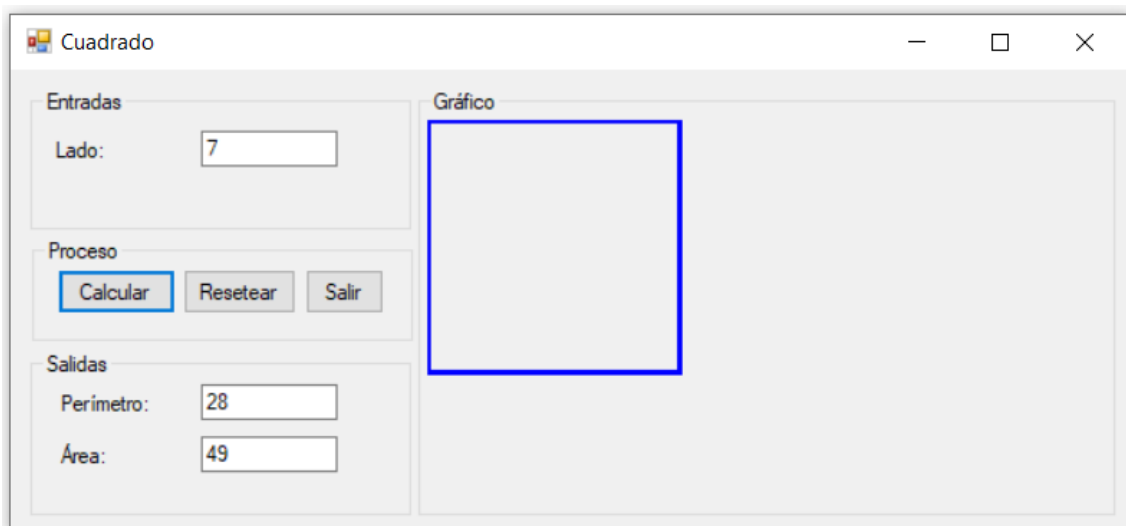


Figura 1.2.6. Ejemplo de la Corrida del Programa.

Caso de Estudio 1.3: Graficar un Círculo y Encontrar el Perímetro y su Área.

A) Problema

Escribir un programa para graficar un círculo, calcular e imprimir el perímetro y su área. (Ver Figura 1.3.1)

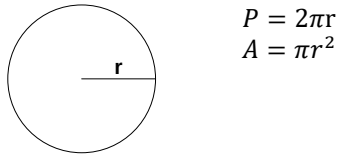


Figura 1.3.1. El círculo y sus fórmulas.

B) Análisis

Claramente, se puede ver que la entrada del problema es el radio del círculo. Hay tres salidas requeridas: el perímetro, el área del círculo y el gráfico de la figura geométrica. Partiendo de un conocimiento básico de geometría, se sabe que hay una relación entre el radio del círculo con el perímetro y con el área de la figura geométrica. Las fórmulas requeridas se incluyen en el requerimiento de los datos.

B.1) Requerimiento de los Datos

Entradas del Problema

radius /* radio del círculo */

Salidas del Problema

perimeter /* perímetro del círculo */
area /* área del círculo */
graph /* objeto que activa el modo gráfico */

Auxiliares del Problema

pen /* Esferográfico de dibujo */

Constantes del Problema

PI = 3.141596 /* relación entre la longitud de una circunferencia y su diámetro */
SF = 20 /* factor de escalamiento */

Fórmulas Relevantes

$P = 2\pi r$ (1) /* Fórmula del perímetro de un círculo */
 $A = \pi r^2$ (2) /* Fórmula del área de un círculo */

B.2) Diagrama de Entrada-Salida

En la Figura 1.3.2 se muestra el Diagrama de Entrada-Salida del programa donde se identifican y se diagraman las entradas, salidas y auxiliares del problema como son: a) Entradas: la variable 'radius'; b) Salidas: la variable 'perímetro', la variable 'área' y el

objeto 'graph'; c) Auxiliares: el objeto 'pen'; d) Constantes: el factor de escalamiento 'SF' y la constante PI.

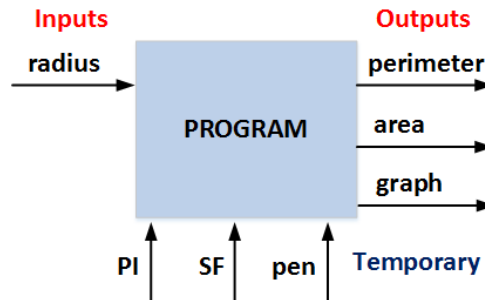


Figura 1.3.2. Diagrama de Entrada-Salida del Programa.

B.3) Diagrama de Clases

El diagrama de clases de esta aplicación se muestra en la Figura 1.3.3 con notación UML. En este diagrama se puede ver que la aplicación está compuesta por seis clases, donde la clase **Program** se compone por la clase **frmCircle** que resulta ser una clase derivada de la clase **Form**. La clase **frmCircle** se compone de un objeto de tipo **CCircle**.

La clase **CCircle** se compone de un objeto de tipo **Graphics** llamado **mGraph**, que maneja el modo gráfico de la aplicación y de un objeto de tipo **Pen** llamado **mPen**, para dibujar sobre un lienzo representado por un **pictureBox** cuyo identificador es **picCanvas**. Además, se tiene una constante llamada **SF** (scale factor) de tipo float que permite realizar un acercamiento (Zoom-In) y un alejamiento (Zoom-Out) sobre la figura geométrica. Los otros atributos que se tienen en la clase **CCircle** son el radio, el perímetro y el área representados por los datos miembros **mRadius**, **mPerimeter** y **mArea** que son de tipo **float**.

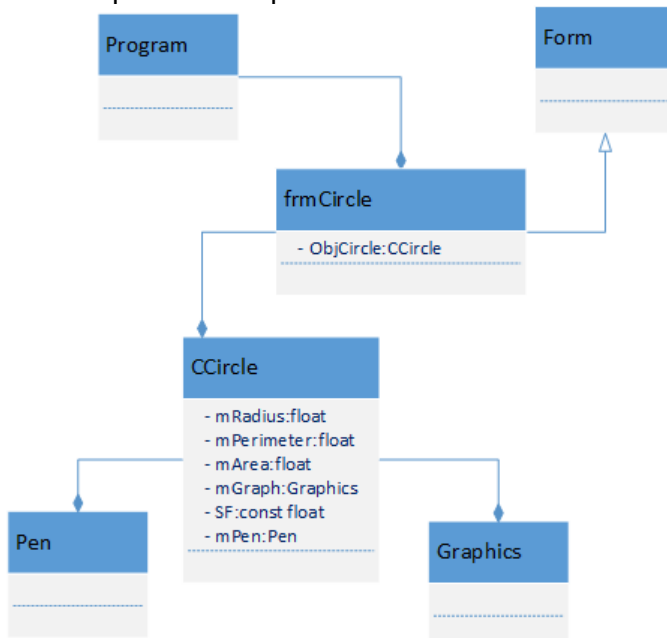


Figura 1.3.3. Diagrama de clases del Caso de Estudio 1.3.

C) Diseño

Una vez que se conoce las entradas y las salidas del problema, se deben listar los pasos necesarios para resolver el problema, es decir, el algoritmo.

C.1) Algoritmos

3.1.1. Algoritmo de la Función *ReadData()*

1. Leer el radio del círculo.

3.1.2. Algoritmo de la Función *PerimeterCircle()*

1. Calcular el perímetro.
 - 1.1. Asignar el producto de dos veces PI por el radio a la variable 'perimeter'.

3.1.3. Algoritmo de la Función *AreaCircle()*

1. Calcular el área.
 - 1.1. Asignar el producto de PI por el cuadrado del radio a la variable 'area'.

3.1.4. Algoritmo de la Función *PrintData()*

1. Imprimir el valor de la variable 'perimeter' y de la variable 'area'.

3.1.5. Algoritmo de la Función *PlotShape()*

1. Graficar un círculo.
 - 1.1. Utilizar el objeto 'Graph' con la función DrawEllipse para graficar con la pluma 'Pen' un círculo.

D) Implementación

Para implementar la solución, se deben escribir los algoritmos como una **Aplicación para Windows** con **C# .NET** que contenga toda la información necesaria para completar la solución de dicho problema y pueda la aplicación ser ejecutada desde Visual Studio .NET.

Para comenzar la construcción de esta aplicación, se debe seleccionar la opción de Crear un proyecto y la Plantilla (template) correspondiente a la **Aplicación de Windows Forms** con el **lenguaje C#**. En el cuadro de diálogo configure su proyecto con el nombre de **WinAppCircle** y luego guarde el mismo en el disco duro.

Ahora que ya se ha guardado el proyecto, acceda al **Explorador de Soluciones** (Solution Explorer) y modifique el nombre del archivo del formulario a **frmCircle.cs**, cuyo nombre por defecto es **Form1.cs** y que consta dentro de la **Aplicación de Windows Forms** con el nombre de **WinAppCircle**, como se muestra en la Figura 1.3.4. Además, cree la clase llamada **CCircle** con la cual va a trabajar el formulario **frmCircle**, aplicando el concepto de Composición y Agregación.

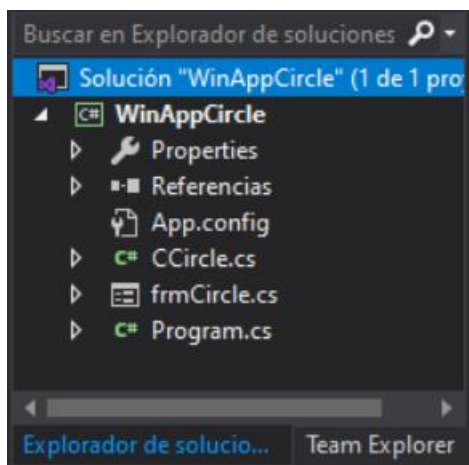


Figura 1.3.4. Solución WinAppCircle.

Se necesita añadir al **Formulario** (Form) tres **Cajas de Grupo** (GroupBoxes), tres **Etiquetas** (Labels), tres **Cajas de Texto** (TextBoxes), tres **Botones** (Buttons), y una **Caja de Imagen** (PictureBox), dispuestos de la siguiente manera: a) La primera **Caja de Grupo** (GroupBox), contiene una **Etiqueta** (Label) y una **Caja de Texto** (TextBox); b) La segunda **Caja de Grupo** (GroupBox), contiene tres **Botones** (Button); c) la tercera **Caja de Grupo** (Group Box), contiene con dos **Etiquetas** (Labels) y dos **Cajas de Texto** (TextBoxes); la cuarta **Caja de Grupo** (GroupBox), contiene un **Caja de Imagen** (PictureBox).

Utilice la Tabla 1.3.1, para modificar las propiedades restantes de cada objeto.

Tabla 1.3.1. Objetos y sus Propiedades.

Objeto	Propiedad	Valor
Form	Name Text StartPosition	frmCircle Círculo CenterScreen
GroupBox	Name Text Font	grbInputs Entradas MS Sans Serif, Bold, 8
Label	Name AutoSize Text	lblRadius True Radio:
TextBox	Name Text	txtRadius Blank
GroupBox	Name Text	grbProcess Blank
Button	Name Text	btnCalculate Calcular
Button	Name Text	btnReset Resetear

Computación Gráfica en C#

Button	Name Text	btnExit Salir
GroupBox	Name Text Font	grbOutputs Salidas MS Sans Serif, Bold, 8
Label	Name AutoSize Text	lblPerimeter True Perímetro:
Label	Name AutoSize Text	lblArea True Área:
TextBox	Name Text Enabled Locked	txtPerimeter Blank False True
TextBox	Name Text Enabled Locked	txtArea Blank False True
GroupBox	Name Text Font	grbCanvas Gráfico MS Sans Serif, Bold, 8
PictureBox	Name	picCanvas

Una vez modificadas las propiedades del formulario y de sus controles queda configurado de la siguiente manera, como se puede ver en la Figura 1.3.5.

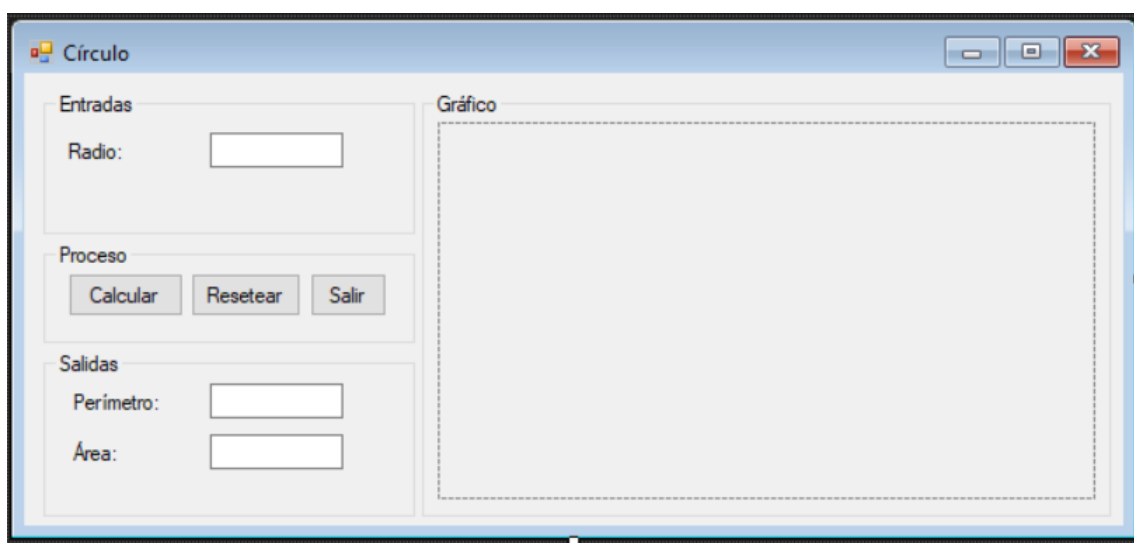


Figura 1.3.5. El Formulario y sus controles modificados sus propiedades.

Esta aplicación aceptará el ingreso de un valor flotante o entero en una **Caja de Texto (TextBox)** y desplegará los resultados en otras dos **Cajas de Texto (TextBoxes)** y el gráfico de la figura se desplegará en una Caja de Imagen (PictureBox).

La clase **CCircle** tiene los siguientes datos miembro: a) **mRadius**, que representa al radio del círculo que es de tipo **float**; b) **mPerimeter**, **mArea**, que representan al perímetro y al área del rectángulo que son también de tipo **float**; c) un objeto **mGraph** de tipo Graphics; d) un esferográfico de dibujo llamado **mPen** de tipo Pen; e) una constante de tipo float llamada **SF** definida como un factor de escalamiento (scale factor).

Esta clase también posee un grupo de métodos de acuerdo a las acciones que puede hacer y que básicamente serán funciones para poder representar y operar un círculo en el plano, entre las más importantes están las siguientes:

- a) **Constructor sin argumentos.** Este método inicializa los primeros tres datos miembros de la clase **CCircle** con el valor de cero.

```
// Constructor sin parámetros.
public CCircle()
{
    mRadius = 0.0f; mPerimeter = 0.0f; mArea = 0.0f;
}
```

- b) **Función miembro ReadData.** Esta función permite leer el valor del radio del círculo ingresado en una **Caja de Texto**. En este caso es necesario realizar un casting o conversión de tipo de datos de String a **float** utilizando la función **float.Parse()** y además se utiliza la función **try-catch** para el manejo y control de excepciones, en caso de que el usuario en lugar de ingresar números ingrese letras por error.

```
// Función que lee el radio del círculo.
public void ReadData(TextBox txtRadius)
{
    try
    {
        mRadius = float.Parse(txtRadius.Text);
    }
    catch
    {
        MessageBox.Show("Ingreso no válido...", "Mensaje de error");
    }
}
```

- c) **Función miembro Perimeter.** Esta función permite calcular el perímetro de un círculo, para lo cual se utiliza una fórmula matemática para poder realizar de esta manera los respectivos cálculos.

```
// Función que calcula el perímetro del círculo.
public void PerimeterCircle()
{
    mPerimeter = 2 * (float)Math.PI * mRadius;
}
```

- d) **Función miembro Area.** Esta función permite calcular el área de un círculo, para lo cual se utiliza una fórmula matemática para poder realizar de esta manera los respectivos cálculos.

```
// Función que calcula el área del círculo.
public void AreaCircle()
{
    mArea = (float)Math.PI * (float)Math.Pow(mRadius, 2);
}
```

- e) **Función miembro PrintData.** Esta función permite imprimir en cajas de texto los valores del perímetro y del área del círculo calculados, para lo cual es necesario realizar un casting o conversión de tipo **float** a String, utilizando la función **ToString()**.

```
// Función que imprime el perímetro y el área del círculo.
public void PrintData(TextBox txtPerimeter, TextBox txtArea)
{
    txtPerimeter.Text = mPerimeter.ToString();
    txtArea.Text = mArea.ToString();
}
```

- f) **Función miembro InitializeData.** Esta función permite inicializar los datos miembro y los controles del formulario con valores por defecto, como en el caso de las **Cajas de Texto** se inicializan con el valor de [BLANK] utilizando los símbolos de comillas dobles (""). Para inicializar la **Caja de Imagen** llamada **picCanvas** se utiliza la función **Refresh** que significa refrescar o borrar.

```
// Función que inicializa los datos y controles del círculo.
public void InitializeData(TextBox txtRadius, TextBox txtPerimeter,
                          TextBox txtArea, PictureBox picCanvas)
{
    mRadius = 0.0f; mPerimeter = 0.0f; mArea = 0.0f;

    txtRadius.Text = ""; txtPerimeter.Text = ""; txtArea.Text = "";
    // Mantiene el cursor titilando en una caja de texto.
    txtRadius.Focus();
    picCanvas.Refresh();
}
```

- g) **Función miembro PlotShape.** Esta función permite graficar un círculo, para lo cual en el objeto **mGraph** se asigna la funcionalidad de crear gráficos de la **Caja de Imagen** llamada **picCanvas**. Luego se crea una pluma de color azul con una punta de ancho 3 conocida como **Brush**. Finalmente, se dibuja un círculo en las coordenadas (0,0) y con los valores del diámetro asignados al **ancho** (width) y al **largo** (height) de la elipse multiplicados por un factor de escalamiento, para que la figura no sea muy pequeña.

```
// Función que grafica un círculo.
public void PlotShape(PictureBox picCanvas)
{
    mGraph = picCanvas.CreateGraphics();
    mPen = new Pen(Color.Blue, 3);
    // Graficar un círculo en base a una elipse.
    mGraph.DrawEllipse(mPen, 0, 0, 2 * mRadius * SF, 2 * mRadius * SF);
}
```

- h) **Función miembro CloseForm.** Esta función permite cerrar un formulario utilizando la función **Close()**, que es llamada por el objeto **ObjForm** el cual es un parámetro de la función que recibe un objeto de tipo **Form**.

```
// Función que cierra un formulario.
public void CloseForm(Form ObjForm)
```

```
{  
    ObjForm.Close();  
}
```

Tip de Programación

Se utiliza la función **float.Parse()** para convertir un tipo de dato carácter o cadena a tipo flotante.

Se utiliza la función **ToString()** para convertir un tipo de dato numérico a tipo carácter o cadena.

La constante **PI** cuyo valor es aproximadamente 3.14159, se la utiliza directamente desde la clase **Math**, que es una clase dedicada a las funciones matemáticas.

La implementación de la clase **CCircle**, se presenta a continuación en la Tabla 1.3.2, que contiene datos miembro y funciones miembro que realizan diferentes operaciones.

Tabla 1.3.2. Código de la clase **CCircle** del programa.

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
using System.Drawing;  
using System.Windows.Forms;  
  
namespace WinAppCircle  
{  
    class CCircle  
    {  
        // Datos Miembro (Atributos).  
  
        // Radio del círculo.  
        private float mRadius;  
        // Perímetro del círculo.  
        private float mPerimeter;  
        // Área del círculo.  
        private float mArea;  
        // Objeto que activa el modo gráfico.  
        private Graphics mGraph;  
        // Constante scale factor (Zoom In/Zoom Out).  
        private const float SF = 20;  
        // Objeto bolígrafo que dibuja o escribe en un lienzo (canvas).  
        private Pen mPen;  
  
        // Funciones Miembro (Métodos).  
  
        // Constructor sin parámetros.  
        public CCircle()  
        {  
            mRadius = 0.0f; mPerimeter = 0.0f; mArea = 0.0f;  
        }  
  
        // Función que lee el radio del círculo.  
        public void ReadData(TextBox txtRadius)  
        {  
            try  
            {  
                mRadius = float.Parse(txtRadius.Text);  
            }  
            catch
```

```

        {
            MessageBox.Show("Ingreso no válido...",
                            "Mensaje de error");
        }
    }

    // Función que calcula el perímetro del círculo.
    public void PerimeterCircle()
    {
        mPerimeter = 2 * (float)Math.PI * mRadius;
    }

    // Función que calcula el área del círculo.
    public void AreaCircle()
    {
        //mArea = (float)Math.PI * mRadius * mRadius;
        mArea = (float)Math.PI * (float)Math.Pow(mRadius, 2);
    }

    // Función que imprime el perímetro y el área del círculo.
    public void PrintData(TextBox txtPerimeter, TextBox txtArea)
    {
        txtPerimeter.Text = mPerimeter.ToString();
        txtArea.Text = mArea.ToString();
    }

    // Función que inicializa los datos y controles del círculo.
    public void InitializeData(TextBox txtRadius, TextBox txtPerimeter,
                              TextBox txtArea, PictureBox picCanvas)
    {
        mRadius = 0.0f; mPerimeter = 0.0f; mArea = 0.0f;

        txtRadius.Text = ""; txtPerimeter.Text = ""; txtArea.Text = "";
        // Mantiene el cursor titilando en una caja de texto.
        txtRadius.Focus();
        picCanvas.Refresh();
    }

    // Función que grafica un círculo.
    public void PlotShape(PictureBox picCanvas)
    {
        mGraph = picCanvas.CreateGraphics();
        mPen = new Pen(Color.Blue, 3);
        // Graficar un círculo en base a una elipse.
        mGraph.DrawEllipse(mPen, 0, 0, 2 * mRadius * SF, 2 * mRadius * SF);
    }

    // Función que cierra un formulario.
    public void CloseForm(Form ObjForm)
    {
        ObjForm.Close();
    }
}

```

La implementación de la clase *frmCircle*, se presenta a continuación en la Tabla 1.3.3, que es la clase que representa a la interfaz gráfica de usuario de la figura geométrica conocida como **Círculo**.

Tabla 1.3.3: Código de la clase *frmCircle* del programa.

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace WinAppCircle
{
    public partial class frmCircle : Form
    {
        // Definición de un objeto de tipo CCircle.
        private CCircle ObjCircle = new CCircle();

        public frmCircle()
        {
            InitializeComponent();
        }

        private void frmCircle_Load(object sender, EventArgs e)
        {
            // Inicialización de los datos y controles.
            // Llamada a la función InitializeData.
            ObjCircle.InitializeData(txtRadius, txtPerimeter,
                                    txtArea, picCanvas);
        }

        private void btnCalculate_Click(object sender, EventArgs e)
        {
            // Lectura de datos.
            // Llamada a la función ReadData.
            ObjCircle.ReadData(txtRadius);
            // Cálculo del perímetro de un círculo.
            // Llamada a la función PerimeterCircle.
            ObjCircle.PerimeterCircle();
            // Cálculo del área de un círculo.
            // Llamada a la función AreaCircle.
            ObjCircle.AreaCircle();
            // Impresión de datos.
            // Llamada a la función PrintData.
            ObjCircle.PrintData(txtPerimeter, txtArea);
            // Graficación de un círculo.
            // Llamada a la función PlotShape.
            ObjCircle.PlotShape(picCanvas);
        }

        private void btnReset_Click(object sender, EventArgs e)
        {
            // Inicialización de los datos y controles.
            // Llamada a la función InitializeData.
            ObjCircle.InitializeData(txtRadius, txtPerimeter,
                                    txtArea, picCanvas);
        }

        private void btnExit_Click(object sender, EventArgs e)
        {
            // Cierre de un formulario.
            // Llamada a la función CloseForm.
            ObjCircle.CloseForm(this);
        }
    }
}
```

```
}  
}  
}
```

La clase **frmCircle** tiene como datos miembro un objeto de tipo **CCircle** llamado **ObjCircle**, el cual es el encargado de acceder a las funciones públicas de la clase **CCircle** para realizar diferentes cálculos y operaciones.

En el evento **Load** del formulario, cuyo procedimiento se llama **frmCircle_Load()**, se utiliza el objeto **ObjCircle**, para llamar a la función pública **InitializeData()**, que se encarga de inicializar los datos y controles del formulario. En el evento **Click** del botón **btnCalculate**, cuyo procedimiento se llama **btnCalculate_Click()**, se utiliza el objeto **ObjCircle**, para llamar a las funciones públicas: **ReadData()**, **PerimeterCircle()**, **AreaCircle()**, **PrintData()** y **PlotShape()**. En el evento **Click** del botón **btnReset**, cuyo procedimiento se llama **btnReset_Click()**, se utiliza el objeto **ObjCircle**, para llamar a la función pública **InitializeData()**. En el evento **Click** del botón **btnClose**, cuyo procedimiento se llama **btnClose_Click()**, se utiliza el objeto **ObjCircle**, para llamar a la función pública **CloseForm()**, que permite cerrar el formulario.

E) Pruebas

Para correr la aplicación basta con presionar la **tecla F5** o el **botón Start** de ejecución de la barra de menús de **Visual Studio .NET**. La Figura 1.1.11 muestra un ejemplo de la corrida de este programa, pero para saber que los resultados del problema son correctos, se debe examinar los resultados del programa cuidadosamente para asegurarnos que tengan sentido. El ejemplo mostrado de la salida del programa proporciona una buena prueba de la solución porque es relativamente fácil calcular el área y el perímetro manualmente o utilizando una calculadora. Para un valor de radio igual a 4 se puede observar la gráfica de la figura geométrica en la **Caja de Imagen** llamada **picCanvas**. El perímetro deberá ser dos veces el radio multiplicado por el valor de PI, que también es un valor fácil de calcular manualmente o utilizando una calculadora. El área deberá ser igual al producto de PI por el valor del radio elevado al cuadrado, que también es un valor fácil de calcular manualmente o utilizando una calculadora. Para verificar que el programa funciona adecuadamente, se deberá ingresar unos pocos valores de prueba correspondientes a los radios de algunos círculos. No se necesita probar más que con unos pocos casos de prueba para verificar que este programa está correcto.

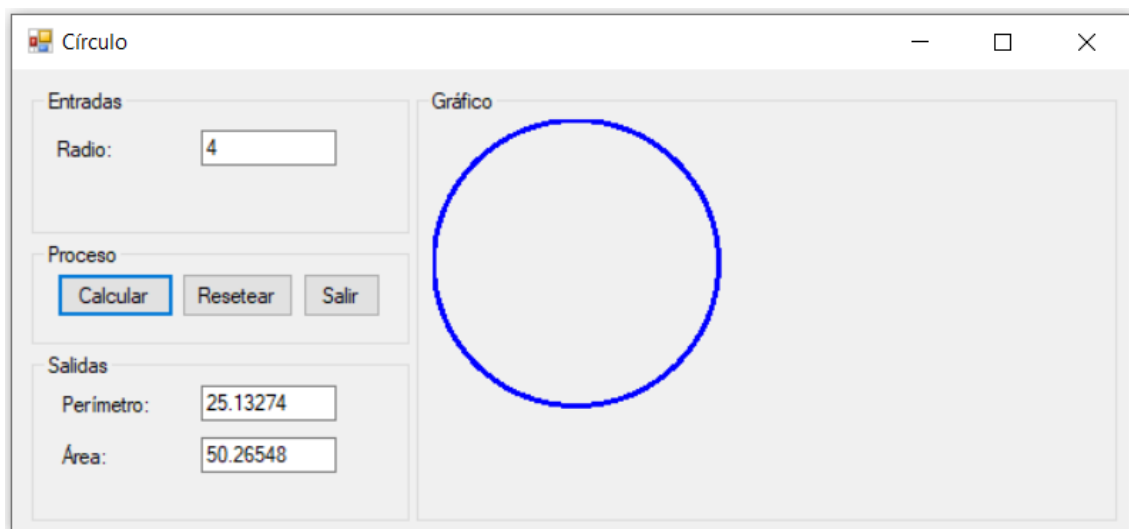


Figura 1.3.6. Ejemplo de la Corrida del Programa.

Caso de Estudio 1.4: Graficar un grupo de líneas utilizando el ratón.

A) Problema

Escribir un programa para graficar un grupo de líneas continuas, donde cada línea se dibuja en función de dos puntos, utilizando el clic izquierdo del ratón. La línea inicial parte desde el origen $O(0,0)$ y la segunda línea comienza en donde terminó la primera, la tercera línea comienza en donde terminó la segunda y así sucesivamente. Además, se debe considerar utilizar dos círculos pequeños para representar graficamente los dos puntos que tiene cada línea. (Ver Figura 1.4.1).

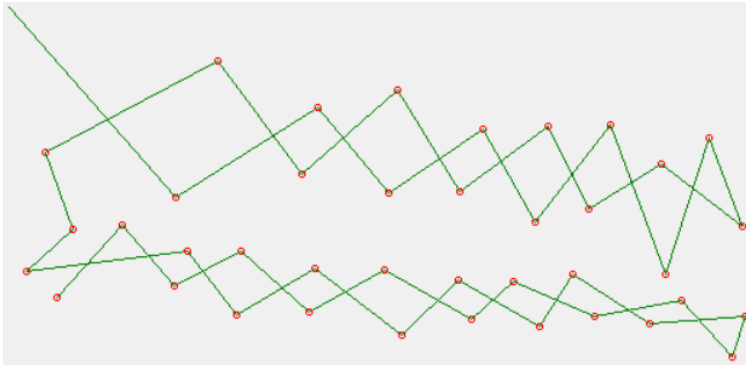


Figura 1.4.1. El Triángulo y sus fórmulas.

B) Análisis

Claramente, se puede ver que las entradas del problema son los dos puntos que conforman la línea, es decir, el punto inicial y el punto final. Hay una sola salida requerida que es un objeto de tipo Graphics que permite dibujar una línea uniendo dos puntos y dibujar un círculo en cada punto de la línea. Los datos auxiliares del problema son los mismos dos puntos que se pueden reutilizar para graficar una nueva línea y además se necesitan dos bolígrafos para el dibujo.

B.1) Requerimiento de los Datos

Entradas del Problema

StartPoint	/* objeto de tipo Point */
EndPoint	/* objeto de tipo Point */

Salidas del Problema

graph	/* objeto que activa el modo gráfico y que se lo utiliza para dibujar una línea y un círculo */
-------	---

Auxiliares del Problema

StartPoint	/* objeto de tipo Point */
EndPoint	/* objeto de tipo Point */
LinePen	/* bolígrafo utilizado para dibujar una línea */
EllipsePen	/* bolígrafo utilizado para dibujar un círculo */

B.2) Diagrama de Entrada-Salida

En la Figura 1.4.2 se muestra el diagrama de Entrada-Salida del programa donde se identifican y se diagraman las entradas y salidas del problema como son: a) Entradas: los objetos StartPoint y EndPoint de tipo Point; b) Salidas: un objeto graph de tipo Graphics; c) Auxiliares: los objetos StartPoint y EndPoint de tipo Point y los bolígrafos para dibujo LinePen y EllipsePen de tipo Pen.

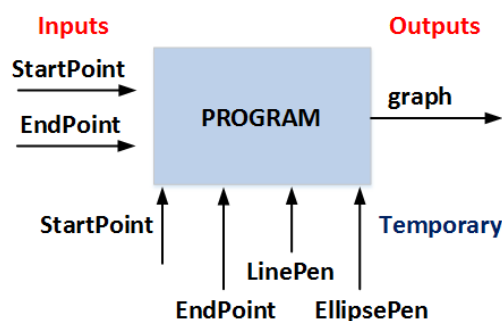


Figura 1.4.2. Diagrama de Entrada-Salida del Programa.

B.3) Diagrama de Clases

El diagrama de clases de esta aplicación se muestra en la Figura 1.4.3 con notación UML. En este diagrama se puede ver que la aplicación está compuesta por seis clases, donde la clase **Program** se compone por la clase **frmLines** que resulta ser una clase derivada de la clase **Form**. La clase **frmLines** se compone de un objeto de tipo **CLine**.

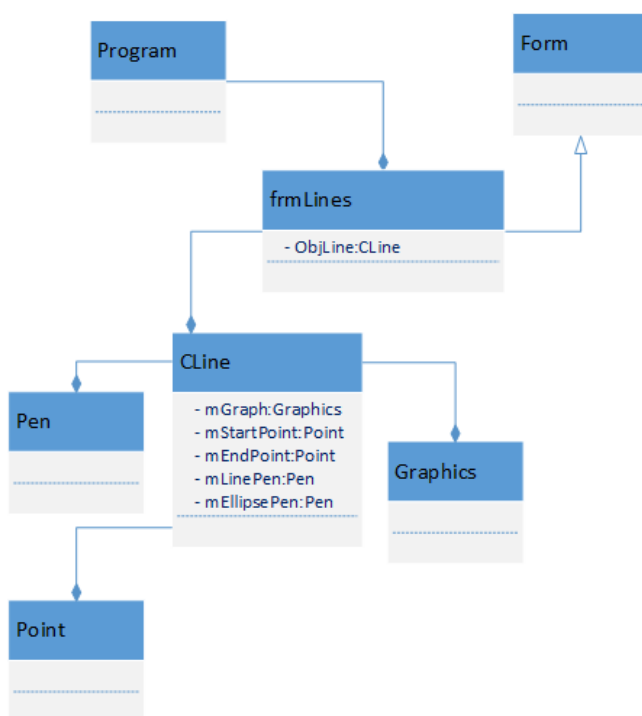


Figura 1.4.3. Diagrama de clases del Caso de Estudio 1.4.

La clase **CLine** se compone de un objeto de tipo **Graphics** llamado **mGraph**, que maneja el modo gráfico de la aplicación y de dos objetos de tipo **Pen** llamados **mLinePen**, y **mEllipsePen** para dibujar sobre un lienzo representado por un **pictureBox** cuyo identificador es **picCanvas**. Los otros atributos que se tienen en la

clase **CLine** son los puntos que representan a una línea llamados **mStartPoint** y **mEndPoint** de tipo Point.

C) Diseño

Una vez que se conoce las entradas y las salidas del problema, se deben listar los pasos necesarios para resolver el problema, es decir, el algoritmo.

C.1) Algoritmos

3.1.1. Algoritmo de la Función *InitializeData()*

1. Inicializar los el punto inicial y final con el valor de cero en las coordenadas 'x' e 'y'.
2. Borrar el contenido de la caja de imagen picCanvas.
3. Asignar con el valor booleano de false a la caja de verificación chkPoint.

3.1.1. Algoritmo de la Función *DrawLineMouse()*

1. Si se ha seleccionado el botón izquierdo del ratón, entonces:
 - 1.1. Asignar la funcionalidad de crear gráficos al objeto mGraph.
 - 1.2. Asignar el valor del punto final al punto inicial.
 - 1.3. Asignar al punto final el valor de las coordenadas del ratón en 'x' e 'y'.
 - 1.4. Dibujar una línea desde el punto actual hasta el nuevo punto.
 - 1.5. Dibujar un círculo para representa a un punto en el plano.
 - 1.6. Desechar los dos objetos bolígrafo de tipo Pen.
 - 1.7. Desechar el objeto graph de tipo Graphics.

D) Implementación

Para implementar la solución, se deben escribir los algoritmos como una **Aplicación para Windows** con **C# .NET** que contenga toda la información necesaria para completar la solución de dicho problema y pueda la aplicación ser ejecutada desde Visual Studio .NET.

Para comenzar la construcción de esta aplicación, se debe seleccionar la opción de Crear un proyecto y la Plantilla (template) correspondiente a la **Aplicación de Windows Forms** con el **lenguaje C#**. En el cuadro de diálogo configure su proyecto con el nombre de WinAppLines y luego guarde el mismo en el disco duro.

Ahora que ya se ha guardado el proyecto, acceda al **Explorador de Soluciones** (Solution Explorer) y modifique el nombre del archivo del formulario a **frmLines.cs**, cuyo nombre por defecto es **Form1.cs** y que consta dentro de la **Aplicación de Windows Forms** con el nombre de **WinAppLines**, como se muestra en la Figura 1.4.4. Además, cree la clase llamada **CLine** con la cual va a trabajar el formulario **frmLines**, aplicando el concepto de Composición y Agregación.

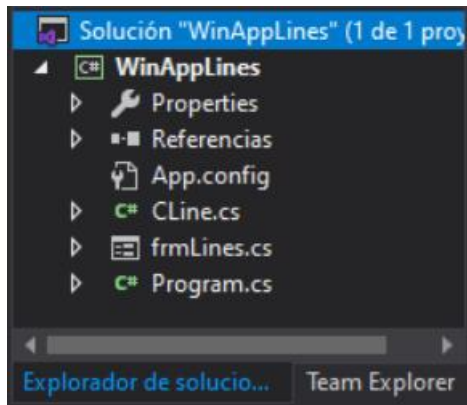


Figura 1.4.4. Solución WinAppLines.

Se necesita añadir al **Formulario** (Form) dos **Cajas de Grupo** (GroupBoxes), dos **Botones** (Buttons) y una **Caja de Imagen** (PictureBox), dispuestos de la siguiente manera: a) La primera **Caja de Grupo** (GroupBox), contiene dos **Botones** (Button); b) la segunda **Caja de Grupo** (GroupBox), contiene un **Caja de Imagen** (PictureBox).

Utilice la Tabla 1.4.1, para modificar las propiedades restantes de cada objeto.

Tabla 1.4.1. Objetos y sus Propiedades.

Objeto	Propiedad	Valor
Form	Name Text StartPosition	frmLines Líneas CenterScreen
GroupBox	Name Text	grbProcess Proceso
Button	Name Text	btnReset Resetear
Button	Name Text	btnExit Salir
GroupBox	Name Text Font	grbCanvas Gráfico MS Sans Serif, Bold, 8
PictureBox	Name	picCanvas

Una vez modificadas las propiedades del formulario y de sus controles queda configurado de la siguiente manera, como se puede ver en la Figura 1.4.5.

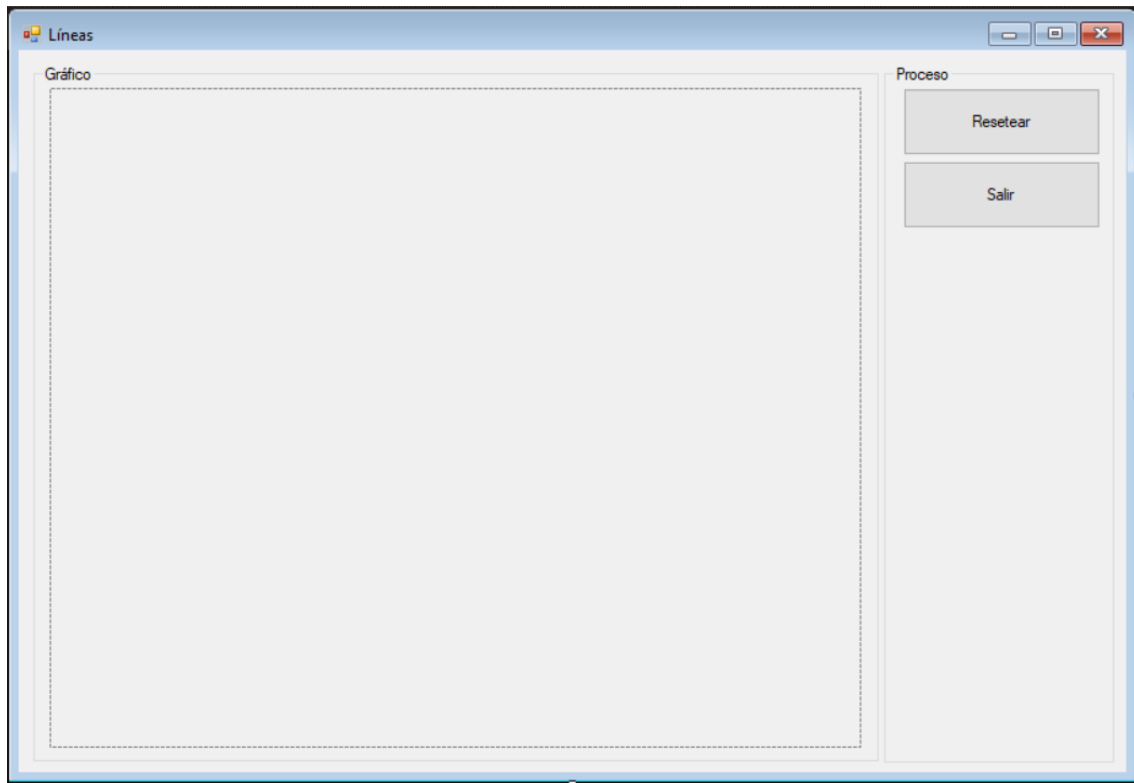


Figura 1.4.5. El Formulario y sus controles modificados sus propiedades.

Esta aplicación desplegará los gráficos de las líneas consecutivas en una **Caja de Imagen** (PictureBox).

La clase **CLine** tiene los siguientes datos miembro: a) un objeto **mGraph** de tipo Graphics; b) Dos objetos de tipo Point llamados **mStartPoint** y **mEndPoint**, que representan a dos puntos en el plano; c) Dos bolígrafos de tipo Pen llamados **mLinePen** y **mEllipsePen** para graficar los dos puntos que conforman una línea con círculos.

Esta clase también posee un grupo de métodos de acuerdo a las acciones que puede hacer y que básicamente serán funciones para poder representar y operar un triángulo en el plano, entre las más importantes están las siguientes:

- a) **Constructor sin argumentos.** Este método inicializa los primeros objetos de tipo Point de la clase **CLine** con el valor de cero en sus coordenadas 'x' e 'y'.

```
// Constructor sin parámetros.
public CLine()
{
    mStartPoint = new Point(0, 0);
    mEndPoint = new Point(0, 0);
}
```

- b) **Función miembro InitializeData.** Esta función permite inicializar los datos miembro y los controles del formulario con valores por defecto. Para inicializar la **Caja de Imagen** llamada **picCanvas** se utiliza la función **Refresh** que significa refrescar o borrar.

```
// Función que inicializa los datos y controles de la clase.
public void InitializeData(PictureBox picCanvas, CheckBox chkPoint)
{
```

```
mStartPoint.X = 0; mStartPoint.Y = 0;
mEndPoint.X = 0; mEndPoint.Y = 0;

picCanvas.Refresh();
chkPoint.Checked = false;
}
```

- c) **Función miembro DrawLineMouse.** Esta función permite dibujar una línea con el clic izquierdo del ratón (mouse), para lo cual se valida que se haya seleccionado dicho botón y luego se procede a crear dos bolígrafos de tipo Pen, uno de color verde para dibujar las líneas y otro de color azul para dibujar los círculos que representan a los dos puntos que conforman una línea. Constantemente, se actualiza el valor del punto inicial con el valor del punto final y el valor del punto final con el valor de las coordenadas 'x' e 'y' del ratón, para poder graficar líneas continuas.

```
// Función que permite dibujar una línea con el clic izquierdo
// del ratón.
public void DrawLineMouse(MouseEventArgs e, PictureBox picCanvas,
                           CheckBox chkPoint)
{
    // Si se ha seleccionado el botón izquierdo del ratón, entonces
    if (e.Button == MouseButtons.Left)
    {
        // Asignar la funcionalidad de crear gráficos al objeto mGraph.
        mGraph = picCanvas.CreateGraphics();

        // Crear dos bolígrafos
        mLinePen = new Pen(Color.Green, 1);
        mEllipsePen = new Pen(Color.Red, 1);

        // Asignar el punto final al punto inicial.
        mStartPoint = mEndPoint;
        // Asignar al punto final la coordenada del ratón.
        mEndPoint = new Point(e.X, e.Y);

        // Dibujar una línea desde el punto actual hasta
        // el nuevo punto.
        mGraph.DrawLine(mLinePen, mStartPoint, mEndPoint);

        // Dibujar un círculo para representar un punto.
        mGraph.DrawEllipse(mEllipsePen, e.X - 2, e.Y - 2, 4, 4);

        // Desechar los objetos
        mLinePen.Dispose();
        mEllipsePen.Dispose();
        mGraph.Dispose();
    }
}
```

- d) **Función miembro CloseForm.** Esta función permite cerrar un formulario utilizando la función **Close()**, que es llamada por el objeto ObjForm el cual es un parámetro de la función que recibe un objeto de tipo **Form**.

```
// Función que cierra un formulario.
public void CloseForm(Form ObjForm)
{
    ObjForm.Close();
}
```

Tip de Programación

Se utiliza la función **float.Parse()** para convertir un tipo de dato carácter o cadena a tipo flotante.

Se utiliza la función **ToString()** para convertir un tipo de dato numérico a tipo carácter o cadena.

La implementación de la clase **CLine**, se presenta a continuación en la Tabla 1.4.2, que contiene datos miembro y funciones miembro que realizan diferentes operaciones.

Tabla 1.4.2. Código de la clase **CLine** del programa.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Drawing;
using System.Windows.Forms;

namespace WinAppLines
{
    class CLine
    {
        // Datos Miembro (Atributos).

        // Objeto que activa el modo gráfico.
        private Graphics mGraph;
        // Objeto Punto que representa al punto inicial de graficación de una línea.
        private Point mStartPoint;
        // Objeto Punto que representa al punto final de graficación de una línea.
        private Point mEndPoint;
        // Objeto bolígrafo que dibuja o escribe una línea.
        Pen mLinePen;
        // Objeto bolígrafo que dibuja o escribe una elipse.
        Pen mEllipsePen;

        // Funciones Miembro (Métodos).

        // Constructor sin parámetros.
        public CLine()
        {
            mStartPoint = new Point(0, 0);
            mEndPoint = new Point(0, 0);
        }

        // Función que inicializa los datos y controles de la clase.
        public void InitializeData(PictureBox picCanvas, CheckBox chkPoint)
        {
            mStartPoint.X = 0; mStartPoint.Y = 0;
            mEndPoint.X = 0; mEndPoint.Y = 0;

            picCanvas.Refresh();
            chkPoint.Checked = false;
        }

        // Función que permite dibujar una línea con el clic izquierdo
        // del ratón.
        public void DrawLineMouse(MouseEventArgs e, PictureBox picCanvas,
                                   CheckBox chkPoint)
        {
            // Si se ha seleccionado el botón izquierdo del ratón, entonces
```

```
        if (e.Button == MouseButtons.Left)
        {
            // Asignar la funcionalidad de crear gráficos al objeto mGraph.
            mGraph = picCanvas.CreateGraphics();

            // Crear dos bolígrafos
            mLinePen = new Pen(Color.Green, 1);
            mEllipsePen = new Pen(Color.Red, 1);

            // Asignar el punto final al punto inicial.
            mStartPoint = mEndPoint;
            // Asignar al punto final la coordenada del ratón.
            mEndPoint = new Point(e.X, e.Y);

            // Dibujar una línea desde el punto actual hasta
            // el nuevo punto.
            mGraph.DrawLine(mLinePen, mStartPoint, mEndPoint);

            // Dibujar un círculo para representar un punto.
            mGraph.DrawEllipse(mEllipsePen, e.X - 2, e.Y - 2, 4, 4);

            // Desechar los objetos
            mLinePen.Dispose();
            mEllipsePen.Dispose();
            mGraph.Dispose();
        }
    }
    // Función que cierra un formulario.
    public void CloseForm(Form ObjForm)
    {
        ObjForm.Close();
    }
}
```

La implementación de la clase *frmLines*, se presenta a continuación en la Tabla 1.4.3, que es la clase que representa a la interfaz gráfica de usuario de la aplicación.

Tabla 1.4.3: Código de la clase *frmLines* del programa.

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace WinAppLines
{
    public partial class frmLines : Form
    {
        // Definición de un objeto de tipo CLine.
        private CLine ObjLine = new CLine();

        public frmLines()
        {
            InitializeComponent();
        }
    }
}
```

```
private void frmLines_Load(object sender, EventArgs e)
{
    // Inicialización de los datos y controles.
    // Llamada a la función InitializeData.
    ObjLine.InitializeData(picCanvas);
}

private void picCanvas_MouseDown(object sender, MouseEventArgs e)
{
    // Graficación de una línea utilizando el ratón.
    // Llamada a la función DrawLineMouse.
    ObjLine.DrawLineMouse(e, picCanvas);
}

private void btnReset_Click(object sender, EventArgs e)
{
    // Inicialización de los datos y controles.
    // Llamada a la función InitializeData.
    ObjLine.InitializeData(picCanvas);
}

private void btnExit_Click(object sender, EventArgs e)
{
    // Cierre de un formulario.
    // Llamada a la función CloseForm.
    ObjLine.CloseForm(this);
}
}
```

La clase *frmLines* tiene como datos miembro un objeto de tipo *CLine* llamado *ObjLine*, el cual es el encargado de acceder a las funciones públicas de la clase *CLine* para realizar diferentes cálculos y operaciones.

En el evento **Load** del formulario, cuyo procedimiento se llama *frmLines_Load()*, se utiliza el objeto *ObjLine*, para llamar a la función pública *InitializeData()*, que se encarga de inicializar los datos y controles del formulario. En el evento **MouseDown** de la **Caja de Imagen** llamada *picCanvas*, cuyo procedimiento se llama *picCanvas_MouseDown()*, se utiliza el objeto *ObjLine*, para llamar a la función pública *DrawLineMouse()*, que permite dibujar líneas continuas. En el evento **Click** del botón *btnClose*, cuyo procedimiento se llama *btnClose_Click()*, se utiliza el objeto *ObjLine*, para llamar a la función pública *CloseForm()*, que permite cerrar el formulario.

Finalmente, para poder trabajar con el evento **MouseDown** de la **Caja de Imagen** llamada *picCanvas*, se debe activar dicho evento en la ventana de propiedades seleccionando el botón de Eventos y haciendo clic en el evento **MouseDown** (ver Figura 1.4.6), lo cual permite registrar este evento en el **Designer** del formulario (ver en la Figura 1.4.7) y luego puede ser utilizado para ser programado.

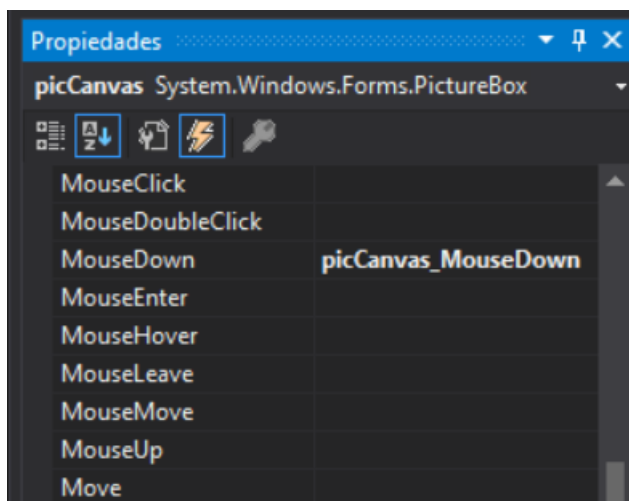


Figura 1.4.6. Activación de eventos del picCanvas.

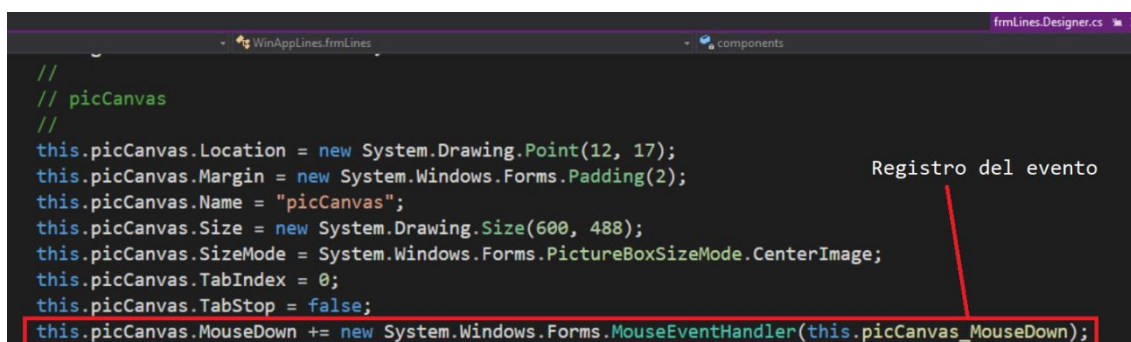


Figura 1.4.7. Registro del evento en el Designer del formulario.

E) Pruebas

Para correr la aplicación basta con presionar la **tecla F5** o el **botón Start** de ejecución de la barra de menús de **Visual Studio .NET**. La Figura 1.4.8 muestra un ejemplo de la corrida de este programa. Para verificar que el programa funciona adecuadamente, se deberá dibujar varias líneas dentro de la **Caja de Imagen** llamada **picCanvas**, haciendo un clic con el botón izquierdo del ratón para definir cada par de puntos que se utilizarán para graficar una línea entre esos dos puntos.

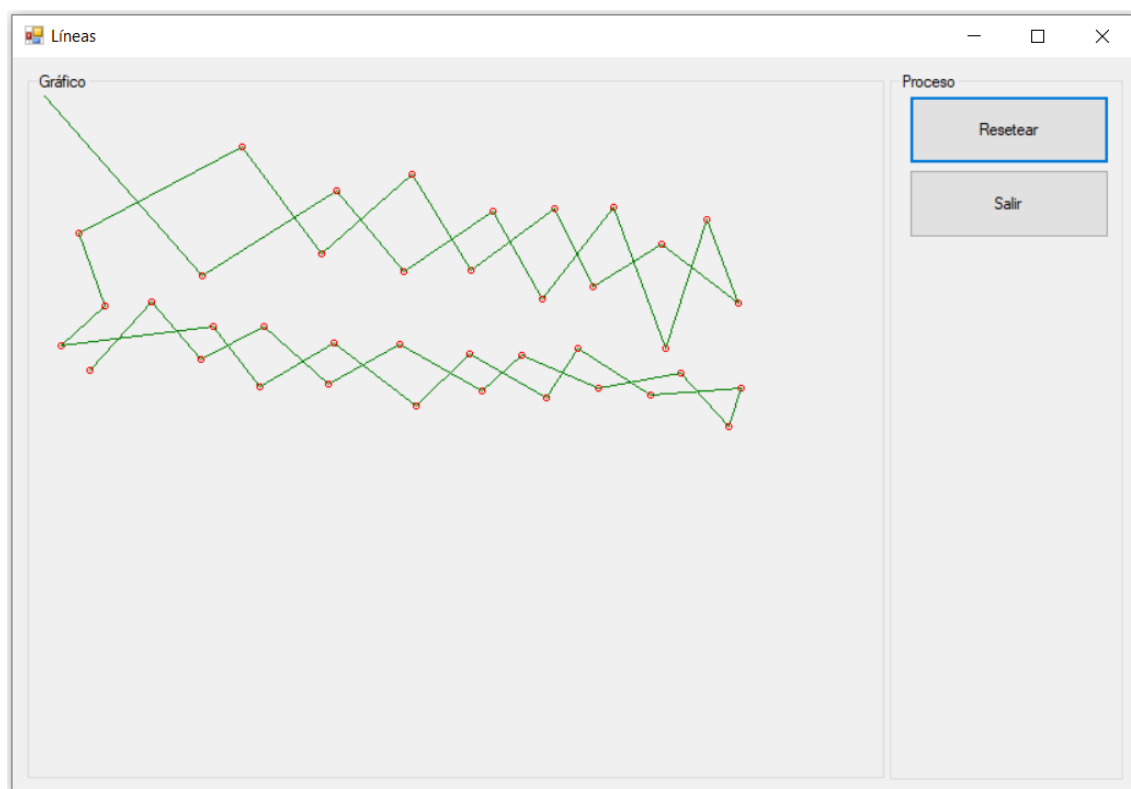


Figura 1.4.8. Ejemplo de la Corrida del Programa.

Caso de Estudio 1.5: Graficar un Triángulo y Encontrar el Perímetro y su Área.

A) Problema

Escribir un programa para graficar un triángulo, calcular e imprimir el perímetro y su área en función del semi-perímetro. Además, validar la existencia de un triángulo. (Ver Figura 1.5.1)

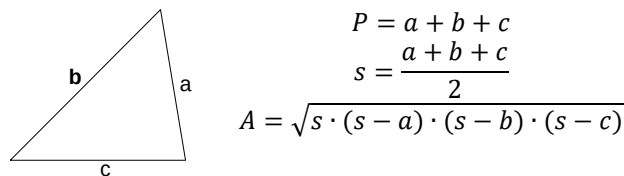


Figura 1.5.1. El Triángulo y sus fórmulas.

B) Análisis

Claramente, se puede ver que las entradas del problema son los tres lados del triángulo. Hay tres salidas requeridas: el perímetro, el área del triángulo y el gráfico de la figura geométrica. Partiendo de un conocimiento básico de geometría, se sabe que hay una relación entre los tres lados del triángulo con el perímetro, con el semi-perímetro y con el área de la figura geométrica. Las fórmulas requeridas se incluyen en el requerimiento de los datos. En este programa referente al triángulo se considera la validación de la existencia de un triángulo cuyo Teorema expresa que un triángulo cualquiera existe si y sólo si la suma de cualquiera de sus dos lados es mayor que el tercer lado: $(a + b > c) \wedge (a + c > b) \wedge (b + c > a)$.

B.1) Requerimiento de los Datos

Entradas del Problema

a	/* lado 1 del triángulo */
b	/* lado 2 del triángulo */
c	/* lado 3 del triángulo */

Salidas del Problema

perimetro	/* perímetro del triángulo */
area	/* área del triángulo */
graph	/* objeto que activa el modo gráfico */

Auxiliares del Problema

s	/* semi-perímetro del triángulo */
Pen	/* Pluma de dibujo */

Constantes del Problema

SF = 10	/* factor de escalamiento */
---------	------------------------------

Fórmulas Relevantes

$$P = a + b + c \quad (1) \quad /* \text{Fórmula del perímetro de un triángulo} */$$

$$s = \frac{a + b + c}{2} \quad (2) \quad /* \text{Fórmula del semi-perímetro de un triángulo} */$$

$$A = \sqrt{s \cdot (s - a) \cdot (s - b) \cdot (s - c)} \quad (3) \quad /* \text{Fórmula del área de un triángulo (Fórmula de Herón)} */$$

B.2) Diagrama de Entrada-Salida

En la Figura 1.5.2 se muestra el diagrama de Entrada-Salida del programa donde se identifican y se diagraman las entradas y salidas del problema como son: a) Entradas: la variable 'a' (lado 1), la variable 'b' (lado 2), la variable 'c' (lado 3); b) Salidas: la variable 'perímetro', la variable 'área' y el objeto 'graph'; c) Auxiliares: la variable 's' (semiperímetro) y el objeto 'pen'; d) Constantes: el factor de escalamiento 'SF'.

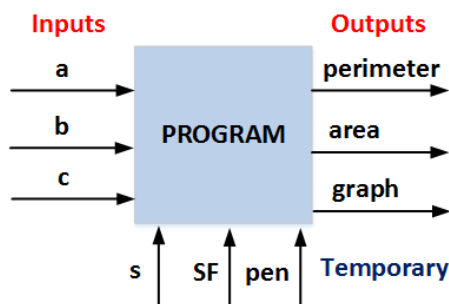


Figura 1.5.2. Diagrama de Entrada-Salida del Programa.

B.3) Diagrama de Clases

El diagrama de clases de esta aplicación se muestra en la Figura 1.5.3 con notación UML. En este diagrama se puede ver que la aplicación está compuesta por seis clases, donde la clase **Program** se compone por la clase **frmTriangle** que resulta ser una clase derivada de la clase **Form**. La clase **frmTriangle** se compone de un objeto de tipo **CTriangle**.

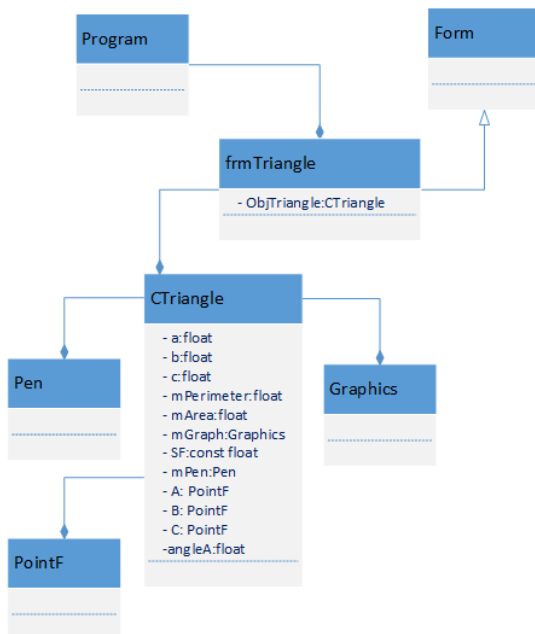


Figura 1.5.3. Diagrama de clases del Caso de Estudio 1.5.

La clase **CTriangle** se compone de un objeto de tipo **Graphics** llamado **mGraph**, que maneja el modo gráfico de la aplicación y de un objeto de tipo **Pen** llamado **mPen**, para dibujar sobre un lienzo representado por un **pictureBox** cuyo identificador es **picCanvas**. Además, se tiene una constante llamada **SF** (scale factor) de tipo **float** que permite realizar un acercamiento (Zoom-In) y un alejamiento (Zoom-Out) sobre la figura geométrica. Los otros atributos que se tienen en la clase **CTriangle** son los tres lados del triángulo, el perímetro y el área representados por los datos miembros **a**, **b**, **c**, **mPerimeter** y **mArea** que son de tipo **float**.

C) Diseño

Una vez que se conoce las entradas y las salidas del problema, se deben listar los pasos necesarios para resolver el problema, es decir, el algoritmo.

C.1) Algoritmos

1.5.1. Algoritmo de la Función *ReadData()*

1. Leer el lado 1 del triángulo.
2. Leer el lado 2 del triángulo.
3. Leer el lado 3 del triángulo.

1.5.1. Algoritmo de la Función *CheckTriangle()*

1. Si la suma de cualquiera de los dos lados es mayor que el tercero, es decir, si se cumple la condición del teorema: $(a + b > c) \wedge (a + c > b) \wedge (b + c > a)$, retornar un valor booleano con verdadero, caso contrario retornar un valor booleano con falso.

1.5.3. Algoritmo de la Función *SemiperimeterTriangle()*

1. Calcular el semi-perímetro.
 - 1.1. Asignar el resultado de la división entre la suma de los tres lados dividido para dos, a la variable 's'.

1.5.4. Algoritmo de la Función *PerimeterTriangle()*

1. Calcular el perímetro.
 - 1.1. Asignar el resultado del producto entre el semi-perímetro por dos a la variable 'perimetro'.

1.5.5. Algoritmo de la Función *AreaTriangle()*

1. Calcular el área.
 - 1.1. Asignar el valor de la raíz cuadrada del semi-perímetro multiplicado por la diferencia entre el semi-perímetro por cada uno de los lados del triángulo, a la variable 'area'.

1.5.6. Algoritmo de la Función *PrintData()*

1. Imprimir el valor de la variable 'perimeter' y de la variable 'area'.

1.5.7. Algoritmo de la Función *PlotShape()*

1. Graficar un triángulo.

- 1.1. Utilizar el objeto 'Graph' con la función DrawLine para graficar con la pluma 'Pen' tres líneas que unen los vértices A, B y C del triángulo.

D) Geometría del Triángulo

Problema: Dados los tres lados de un triángulo, encontrar los valores de los vértices en coordenadas rectangulares y graficar las líneas del triángulo, considerando que uno de sus lados se encuentra sobre el eje de las 'x' y uno de sus vértices es el origen. (Ver Figura 1.5.4 y 1.5.5).

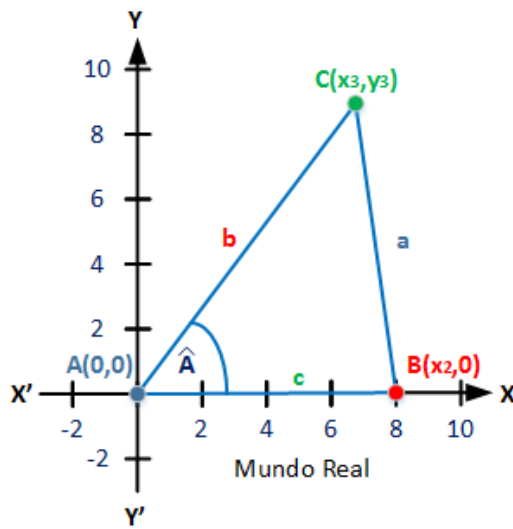


Figura 1.5.4. Triángulo en el mundo real.

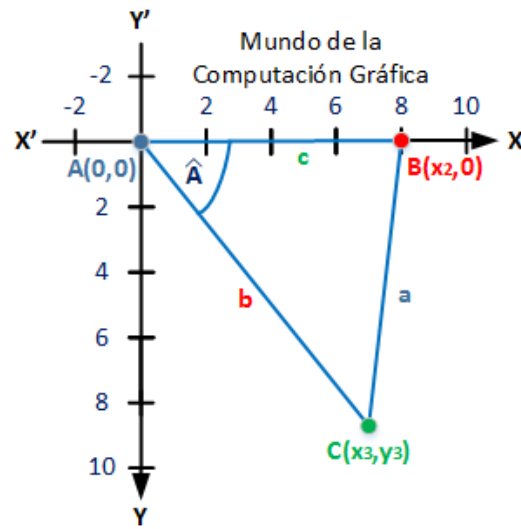


Figura 1.5.5. Triángulo en el mundo de la Computación Gráfica.

Solución:

Dados los siguientes puntos:

$$\begin{aligned} A(0,0) &= P_1(x_1, y_1) \\ B(x_B, 0) &= P_2(x_2, y_2) \\ C(x_C, y_C) &= P_3(x_3, y_3) \end{aligned}$$

Calculamos la distancia **AB**:

$$\begin{aligned} d_{AB} &= d_{P_1P_2} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} = c \\ c &= \sqrt{(x_2 - 0)^2 + (0 - 0)^2} \\ c &= \sqrt{x_2^2} \\ c &= x_2 \end{aligned} \quad (1)$$

Por lo tanto, las coordenadas del punto **P₁** y **P₂** son:

$$A(0,0) = P_1(0,0) \quad (2)$$

$$B(x_B, 0) = P_2(c, 0) \quad (3)$$

Para calcular el valor del ángulo **A**, se utilizando la Ley de Cosenos:

$$\cos(A) = \frac{b^2 + c^2 - a^2}{2bc}$$

$$A = \arccos\left(\frac{b^2 + c^2 - a^2}{2bc}\right)$$

Calculamos el valor de la pendiente **AC**:

$$\begin{aligned} m_{AC} &= \frac{y_3 - y_1}{x_3 - x_1} = tg(A) \\ tg(A) &= \frac{y_3 - 0}{x_3 - 0} \\ tg(A) &= \frac{y_3}{x_3} \end{aligned}$$

Despejamos el valor de y_3 donde obtenemos la siguiente ecuación:

$$y_3 = x_3 \cdot tg(A) \quad (4)$$

Calculamos la distancia **AC**:

$$\begin{aligned} d_{AC} &= d_{P_1P_3} = \sqrt{(x_3 - x_1)^2 + (y_3 - y_1)^2} = b \\ b &= \sqrt{(x_3 - 0)^2 + (y_3 - 0)^2} \\ b &= \sqrt{x_3^2 + y_3^2} \end{aligned} \quad (5)$$

Reemplazamos el valor de y_3 de la ecuación (4) en la ecuación (5) y calculamos el valor de x_3 :

$$\begin{aligned} b &= \sqrt{x_3^2 + (x_3 \cdot tg(A))^2} \\ b &= \sqrt{x_3^2 + x_3^2 \cdot tg^2(A)} \\ b &= \sqrt{x_3^2(1 + tg^2(A))} \\ b &= x_3 \cdot \sqrt{1 + tg^2(A)} \end{aligned}$$

Despejamos el valor de x_3 y luego racionalizamos la expresión:

$$\begin{aligned} x_3 &= \frac{b}{\sqrt{1 + tg^2(A)}} \cdot \frac{\sqrt{1 + tg^2(A)}}{\sqrt{1 + tg^2(A)}} \\ x_3 &= \frac{b \cdot \sqrt{1 + tg^2(A)}}{1 + tg^2(A)} \end{aligned} \quad (6)$$

Reemplazamos el valor de x_3 de la ecuación (6) en la ecuación (4) y calculamos el valor de y_3 :

$$y_3 = \frac{b \cdot \sqrt{1 + tg^2(A)}}{1 + tg^2(A)} \cdot tg(A) \quad (7)$$

Por lo tanto, las coordenadas del punto **P₃** son:

$$C(x_c, y_c) = P_3 \left(\frac{b \cdot \sqrt{1 + tg^2(A)}}{1 + tg^2(A)}, \frac{b \cdot \sqrt{1 + tg^2(A)}}{1 + tg^2(A)} \cdot tg(A) \right) \quad (8)$$

Aplicando las siguientes identidades trigonométricas:

$$\sec^2(A) = 1 + tg^2(A) \quad (9)$$

$$tg(A) = \frac{\text{sen}(A)}{\cos(A)} \quad (10)$$

$$\sec(A) = \frac{1}{\cos(A)} \quad (11)$$

Se simplifica la ecuación (8) de la siguiente manera:

$$x_c = \frac{b \cdot \sqrt{1 + tg^2(A)}}{1 + tg^2(A)} = \frac{b \cdot \sqrt{\sec^2(A)}}{\sec^2(A)} = \frac{b \cdot \sec(A)}{\sec^2(A)} = \frac{b}{\sec(A)} = \frac{b}{\frac{1}{\cos(A)}} = b \cdot \cos(A)$$

$$x_c = b \cdot \cos(A) \quad (12)$$

$$y_c = \frac{b \cdot \sqrt{1 + tg^2(A)}}{1 + tg^2(A)} \cdot tg(A) = x_c \cdot tg(A) = b \cdot \cos(A) \cdot tg(A) = b \cdot \cos(A) \cdot \frac{\text{sen}(A)}{\cos(A)} = b \cdot \text{sen}(A)$$

$$y_c = b \cdot \text{sen}(A) \quad (13)$$

Finalmente, se obtienen las coordenadas del punto P_3 en función de senos y cosenos:

$$C(x_c, y_c) = P_3(b \cdot \cos(A), b \cdot \text{sen}(A)) \quad (14)$$

E) Implementación

Para implementar la solución, se deben escribir los algoritmos como una **Aplicación para Windows** con **C# .NET** que contenga toda la información necesaria para completar la solución de dicho problema y pueda la aplicación ser ejecutada desde Visual Studio .NET.

Para comenzar la construcción de esta aplicación, se debe seleccionar la opción de Crear un proyecto y la Plantilla (template) correspondiente a la **Aplicación de Windows Forms** con el **lenguaje C#**. En el cuadro de diálogo configure su proyecto con el nombre de **WinAppTriangle** y luego guarde el mismo en el disco duro.

Ahora que ya se ha guardado el proyecto, acceda al **Explorador de Soluciones** (Solution Explorer) y modifique el nombre del archivo del formulario a **frmTriangle.cs**, cuyo nombre por defecto es **Form1.cs** y que consta dentro de la **Aplicación de Windows Forms** con el nombre de **WinAppTriangle**, como se muestra en la Figura 1.5.6. Además, cree la clase llamada **CTriangle** con la cual va a trabajar el formulario **frmTriangle**, aplicando el concepto de Composición y Agregación.

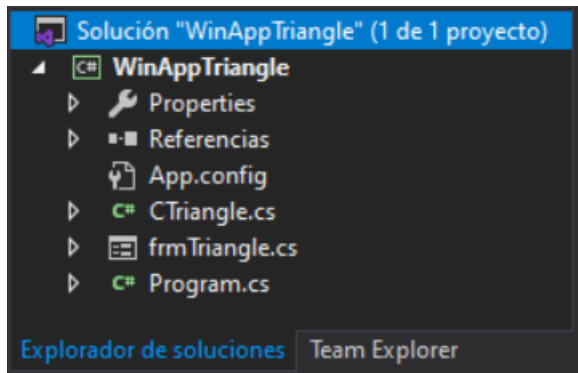


Figura 1.5.6. Solución WinAppTriangle.

Se necesita añadir al **Formulario** (Form) tres **Cajas de Grupo** (GroupBoxes), cinco **Etiquetas** (Labels), cinco **Cajas de Texto** (TextBoxes), tres **Botones** (Buttons), y una **Caja de Imagen** (PictureBox), dispuestos de la siguiente manera: a) La primera **Caja de Grupo** (GroupBox), contiene tres Etiquetas (Label) y tres **Cajas de Texto** (TextBox); b) La segunda **Caja de Grupo** (GroupBox), contiene tres **Botones** (Button); c) la tercera **Caja de Grupo** (Group Box), contiene con dos **Etiquetas** (Labels) y dos **Cajas de Texto** (TextBoxes); la cuarta **Caja de Grupo** (GroupBox), contiene un **Caja de Imagen** (PictureBox).

Utilice la Tabla 1.5.1, para modificar las propiedades restantes de cada objeto.

Tabla 1.5.1. Objetos y sus Propiedades.

Objeto	Propiedad	Valor
Form	Name Text StartPosition	frmTriangle Triángulo CenterScreen
GroupBox	Name Text Font	grbInputs Entradas MS Sans Serif, Bold, 8
Label	Name AutoSize Text	lblSideA True Lado a:
Label	Name AutoSize Text	lblSideB True Lado b:
Label	Name AutoSize Text	lblSideC True Lado c:
TextBox	Name Text	txtSideA Blank
TextBox	Name Text	txtSideA Blank
TextBox	Name Text	txtSideA Blank
GroupBox	Name Text Font	grbProcess Proceso MS Sans Serif, Bold, 8
Button	Name Text	btnCalculate Calcular
Button	Name Text	btnReset Resetear
Button	Name	btnExit

	Text	Salir
GroupBox	Name Text Font	grbOutputs Salidas MS Sans Serif, Bold, 8
Label	Name AutoSize Text	lblPerimeter True Perímetro:
Label	Name AutoSize Text	lblArea True Área:
TextBox	Name Text Enabled Locked	txtPerimeter Blank False True
TextBox	Name Text Enabled Locked	txtArea Blank False True
GroupBox	Name Text Font	grbCanvas Gráfico MS Sans Serif, Bold, 8
PictureBox	Name	picCanvas

Una vez modificadas las propiedades del formulario y de sus controles queda configurado de la siguiente manera, como se puede ver en la Figura 1.5.7.

Figura 1.5.7. El Formulario y sus controles modificados sus propiedades.

Esta aplicación aceptará el ingreso de un valor flotante o entero en tres **Cajas de Texto** (TextBox) y desplegará los resultados en otras dos **Cajas de Texto** (TextBoxes) y el gráfico de la figura se desplegará en una **Caja de Imagen** (PictureBox).

La clase **CTriangle** tiene los siguientes datos miembro: a) **a**, **b** y **c**, que representan a los tres lados del triángulo que son de tipo **float**; b) **mPerimeter**, **mArea**, que representan al perímetro y al área del rectángulo que son también de tipo **float**; c) un objeto **mGraph** de tipo Graphics; d) una pluma de dibujo llamada **mPen** de tipo Pen; e) una constante de tipo float llamada **SF** definida como un factor de escalamiento (scale factor).

Esta clase también posee un grupo de métodos de acuerdo a las acciones que puede hacer y que básicamente serán funciones para poder representar y operar un triángulo en el plano, entre las más importantes están las siguientes:

- a) **Constructor sin argumentos.** Este método inicializa los primeros cinco datos miembros de la clase **CTriangle** con el valor de cero.

```
// Constructor sin parámetros.
public CTriangle()
{
    a = 0.0f; b = 0.0f; c = 0.0f;
    perimeter = 0.0f; area = 0.0f;
}
```

- b) **Función miembro ReadData.** Esta función permite leer los tres valores de los lados del triángulo ingresados en **Cajas de Texto**. En este caso es necesario realizar un casting o conversión de tipo de datos de String a **float** utilizando la función **float.Parse()** y además se utiliza la función **try-catch** para el manejo y control de excepciones, en caso de que el usuario en lugar de ingresar números ingrese letras por error.

```
// Función que lee los tres lados del triángulo.
public void ReadData(TextBox txtSideA,
                    TextBox txtSideB,
                    TextBox txtSideC)
{
    try
    {
        a = float.Parse(txtSideA.Text);
        b = float.Parse(txtSideB.Text);
        c = float.Parse(txtSideC.Text);
    }
    catch
    {
        MessageBox.Show("Ingreso no válido...");
    }
}
```

- c) **Función miembro Semiperimeter.** Esta función permite calcular el semi-perímetro de un triángulo, para lo cual se utiliza una fórmula matemática para poder realizar de esta manera los respectivos cálculos. Esta función al ser privada no puede ser accedida por un objeto sino solo puede ser llamada por otras funciones dentro de la clase **CTriangle**.

```
// Función que calcula el semi-perímetro del triángulo.
private float SemiperimeterTriangle()
{
```

```
        return ((a + b + c) / 2);  
    }
```

- d) **Función miembro Perimeter.** Esta función permite calcular el perímetro de un triángulo en función de su semi-perímetro, para lo cual se utiliza una fórmula matemática para poder realizar de esta manera los respectivos cálculos.

```
// Función que calcula el perímetro del triángulo.  
public void PerimeterTriangle()  
{  
    float s = SemiperimeterTriangle();  
    perimeter = 2 * s;  
}
```

- e) **Función miembro Area.** Esta función permite calcular el área de un triángulo en función de su semi-perímetro, para lo cual se utiliza la fórmula matemática de Herón para poder realizar de esta manera los respectivos cálculos.

```
// Función que calcula el área del triángulo.  
public void AreaTriangle()  
{  
    float s = SemiperimeterTriangle();  
    area = (float)Math.Sqrt(s * (s - a) * (s - b) * (s - c));  
}
```

- f) **Función miembro PrintData.** Esta función permite imprimir en cajas de texto los valores del perímetro y del área del círculo calculados, para lo cual es necesario realizar un casting o conversión de tipo *float* a String, utilizando la función **ToString()**.

```
// Función que imprime el perímetro y el área del triángulo.  
public void PrintData(TextBox txtPerimeter, TextBox txtArea)  
{  
    txtPerimeter.Text = mPerimeter.ToString();  
    txtArea.Text = mArea.ToString();  
}
```

- g) **Función miembro InitializeData.** Esta función permite inicializar los datos miembro y los controles del formulario con valores por defecto, como en el caso de las **Cajas de Texto** se inicializan con el valor de [BLANK] utilizando los símbolos de comillas dobles (""). Para inicializar la **Caja de Imagen** llamada **picCanvas** se utiliza la función **Refresh** que significa refrescar o borrar.

```
// Función que inicializa los datos y controles del triángulo.  
public void InitializeData(TextBox txtSideA, TextBox txtSideB,  
                           TextBox txtSideC, TextBox txtPerimeter,  
                           TextBox txtArea, PictureBox picCanvas)  
{  
    a = 0.0f; b = 0.0f; c = 0.0f;  
    perimeter = 0.0f; area = 0.0f;  
    txtSideA.Text = ""; txtSideB.Text = ""; txtSideC.Text = "";  
    txtPerimeter.Text = ""; txtArea.Text = "";  
    // Mantiene el cursor titilando en una caja de texto.  
    txtSideA.Focus();  
    picCanvas.Refresh();  
}
```

- h) **Función miembro CheckTriangle.** Esta función permite verificar la existencia de un triángulo utilizando un teorema matemático. La función retorna un valor

booleano verdadero si existe el triángulo y retorna un valor booleano falso si no existe el triángulo.

```
// Función que valida la existencia de un triángulo.
public bool CheckTriangle()
{
    if ((a + b > c) && (a + c > b) && (b + c > a))
    {
        return (true);
    }
    else // !((mA + mB > mC) && (mA + mC > mB) && (mB + mC > mA))
        return (false);
}
```

- i) **Función miembro CalculateAngleA.** Esta función permite calcular el **ángulo A** de un triángulo cualquiera, utilizando las leyes trigonométricas de los cosenos.

```
// Función que calcula el ángulo A del triángulo.
private void CalculateAngleA()
{
    angleA = (float)Math.Acos((b * b + c * c - a * a) / (2 * b * c));
}
```

- a) **Función miembro CalculateVertex.** Esta función permite calcular los vértices del triángulo en función del **ángulo A** y utilizando las fórmulas geométricas considerando que uno de los **vértices (A)** está sobre el origen y otro **vértice (B)** sobre el eje de las 'x'.

```
// Función que calcula los valores de los tres vértices del triángulo.
private void CalculateVertex()
{
    A.X = 0.0f; A.Y = 0.0f;
    B.X = c; B.Y = 0.0f;

    CalculateAngleA();

    C.X = b * (float)Math.Cos(angleA);
    C.Y = b * (float)Math.Sin(angleA);
}
```

- b) **Función miembro PlotShape.** Esta función permite graficar un triángulo, para lo cual en el objeto **mGraph** se asigna la funcionalidad de crear gráficos de la **Caja de Imagen** llamada **picCanvas**. Luego se crea una pluma de color azul con una punta de ancho 3 conocida como **Brush**. Finalmente, se dibuja un triángulo en las coordenadas (0,0), utilizando tres vértices multiplicados por un factor de escalamiento y tres líneas que permiten unir los vértices de dos en dos.

```
// Función que grafica un triángulo en base a los tres
// vértices, representados por tres puntos en un plano.
public void PlotShape(PictureBox picCanvas)
{
    mGraph = picCanvas.CreateGraphics();
    mPen = new Pen(Color.Blue, 3);

    CalculateVertex();

    // Graficar las tres líneas que conforman un triángulo.
    mGraph.DrawLine(mPen, A.X * SF, A.Y * SF, B.X * SF, B.Y * SF);
    mGraph.DrawLine(mPen, A.X * SF, A.Y * SF, C.X * SF, C.Y * SF);
    mGraph.DrawLine(mPen, B.X * SF, B.Y * SF, C.X * SF, C.Y * SF);
}
```

```
}
```

- c) **Función miembro CloseForm.** Esta función permite cerrar un formulario utilizando la función **Close()**, que es llamada por el objeto ObjForm el cual es un parámetro de la función que recibe un objeto de tipo **Form**.

```
// Función que cierra un formulario.
public void CloseForm(Form ObjForm)
{
    ObjForm.Close();
}
```

Tip de Programación

Se utiliza la función **float.Parse()** para convertir un tipo de dato carácter o cadena a tipo flotante.

Se utiliza la función **ToString()** para convertir un tipo de dato numérico a tipo carácter o cadena.

La implementación de la clase **CTriangle**, se presenta a continuación en la Tabla 1.5.2, que contiene datos miembro y funciones miembro que realizan diferentes operaciones.

Tabla 1.5.2. Código de la clase **CTriangle** del programa.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Drawing;
using System.Windows.Forms;

namespace WinAppTriangle
{
    class CTriangle
    {
        // Datos Miembro (Atributos).

        // Lado 1 del triángulo.
        private float a;
        // Lado 2 del triángulo.
        private float b;
        // Lado 3 del triángulo.
        private float c;
        // Perímetro del triángulo.
        private float perimeter;
        // Área del triángulo.
        private float area;
        // Objeto que activa el modo gráfico.
        private Graphics mGraph;
        // Objeto bolígrafo que dibuja o escribe en un lienzo (canvas).
        private Pen mPen;
        // Constante scale factor (Zoom In/Zoom Out).
        private const float SF = 10;
        // Objeto Punto que representa al vértice A del triángulo.
        private PointF A;
        // Objeto Punto que representa al vértice B del triángulo.
        private PointF B;
        // Objeto Punto que representa al vértice C del triángulo.
        private PointF C;
        // Ángulo A del triángulo.
        private float angleA;
    }
}
```

```
// Funciones Miembro (Métodos).

// Constructor sin parámetros.
public CTriangle()
{
    a = 0.0f; b = 0.0f; c = 0.0f;
    perimeter = 0.0f; area = 0.0f;
}

// Función que lee los tres lados del triángulo.
public void ReadData(TextBox txtSideA,
                    TextBox txtSideB,
                    TextBox txtSideC)
{
    try
    {
        a = float.Parse(txtSideA.Text);
        b = float.Parse(txtSideB.Text);
        c = float.Parse(txtSideC.Text);
    }
    catch
    {
        MessageBox.Show("Ingreso no válido...");
    }
}

// Función que calcula el semi-perímetro del triángulo.
private float SemiperimeterTriangle()
{
    return ((a + b + c) / 2);
}

// Función que calcula el perímetro del triángulo.
public void PerimeterTriangle()
{
    float s = SemiperimeterTriangle();
    perimeter = 2 * s;
}

// Función que calcula el área del triángulo.
public void AreaTriangle()
{
    float s = SemiperimeterTriangle();
    area = (float)Math.Sqrt(s * (s - a) * (s - b) * (s - c));
}

// Función que imprime el perímetro y el área del triángulo.
public void PrintData(TextBox txtPerimeter, TextBox txtArea)
{
    txtPerimeter.Text = perimeter.ToString();
    txtArea.Text = area.ToString();
}

// Función que inicializa los datos y controles del triángulo.
public void InitializeData(TextBox txtSideA, TextBox txtSideB,
                        TextBox txtSideC, TextBox txtPerimeter,
                        TextBox txtArea, PictureBox picCanvas)
{
    a = 0.0f; b = 0.0f; c = 0.0f;
    perimeter = 0.0f; area = 0.0f;
    txtSideA.Text = ""; txtSideB.Text = ""; txtSideC.Text = "";
}
```

```
        txtPerimeter.Text = ""; txtArea.Text = "";
        // Mantiene el cursor titilando en una caja de texto.
        txtSideA.Focus();
        picCanvas.Refresh();
    }

    // Función que valida la existencia de un triángulo.
    public bool CheckTriangle()
    {
        if ((a + b > c) && (a + c > b) && (b + c > a))
        {
            return (true);
        }
        else // !((mA + mB > mC) && (mA + mC > mB) && (mB + mC > mA))
            return (false);
    }

    // Función que calcula el ángulo A del triángulo.
    private void CalculateAngleA()
    {
        angleA = (float)Math.Acos((b * b + c * c - a * a) / (2 * b * c));
    }

    // Función que calcula los valores de los tres vértices del triángulo.
    private void CalculateVertex()
    {
        A.X = 0.0f; A.Y = 0.0f;
        B.X = c; B.Y = 0.0f;

        CalculateAngleA();

        C.X = b * (float)Math.Cos(angleA);
        C.Y = b * (float)Math.Sin(angleA);
    }

    // Función que grafica un triángulo en base a los tres
    // vértices, representados por tres puntos en un plano.
    public void PlotShape(PictureBox picCanvas)
    {
        mGraph = picCanvas.CreateGraphics();
        mPen = new Pen(Color.Blue, 3);

        CalculateVertex();

        // Graficar las tres líneas que conforman un triángulo.
        mGraph.DrawLine(mPen, A.X * SF, A.Y * SF, B.X * SF, B.Y * SF);
        mGraph.DrawLine(mPen, A.X * SF, A.Y * SF, C.X * SF, C.Y * SF);
        mGraph.DrawLine(mPen, B.X * SF, B.Y * SF, C.X * SF, C.Y * SF);
    }

    // Función que cierra un formulario.
    public void CloseForm(Form ObjForm)
    {
        ObjForm.Close();
    }
}
```

La implementación de la clase *frmTriangle*, se presenta a continuación en la Tabla 1.5.3, que es la clase que representa a la interfaz gráfica de usuario de la figura geométrica conocida como **Triángulo**.

Tabla 1.5.3: Código de la clase *frmTriangle* del programa.

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace WinAppTriangle
{
    public partial class frmTriangle : Form
    {
        // Definición de un objeto de tipo CTriangle.
        private CTriangle ObjCTriangle = new CTriangle();

        public frmTriangle()
        {
            InitializeComponent();
        }

        private void frmTriangle_Load(object sender, EventArgs e)
        {
            // Inicialización de los datos y controles.
            // Llamada a la función InitializeData.
            ObjCTriangle.InitializeData(txtSideA, txtSideB, txtSideC,
                                     txtPerimeter, txtArea, picCanvas);
        }

        private void btnCalculate_Click(object sender, EventArgs e)
        {
            // Declaración de una variable booleana.
            bool Validate;

            // Lectura de datos.
            // Llamada a la función ReadData.
            ObjCTriangle.ReadData(txtSideA, txtSideB, txtSideC);
            // Verificación de la existencia de un triángulo.
            // Llamada a la función CheckTriangle.
            Validate = ObjCTriangle.CheckTriangle();

            // Si el valor de la variable booleana es verdadero,
            // entonces existe y se realizan algunos cálculos.
            if (Validate == true)
            {
                // Cálculo del perímetro de un triángulo.
                // Llamada a la función PerimeterTriangle.
                ObjCTriangle.PerimeterTriangle();
                // Cálculo del área de un triángulo.
                // Llamada a la función AreaTriangle.
                ObjCTriangle.AreaTriangle();
                // Impresión de datos.
                // Llamada a la función PrintData.
                ObjCTriangle.PrintData(txtPerimeter, txtArea);
                // Graficación de un triángulo.
                // Llamada a la función PlotShape.
                ObjCTriangle.PlotShape(picCanvas);
            }
            // Caso Contrario, el triángulo no existe.
        }
    }
}
```



```
        else // !(Validate == true)
        {
            MessageBox.Show("Error...el triángulo no existe.",
                            "Mensaje de error");
        }
    }

    private void btnReset_Click(object sender, EventArgs e)
    {
        // Inicialización de los datos y controles.
        // Llamada a la función InitializeData.
        ObjCTriangle.InitializeData(txtSideA, txtSideB, txtSideC,
                                    txtPerimeter, txtArea, picCanvas);
    }

    private void btnExit_Click(object sender, EventArgs e)
    {
        // Cierre de un formulario.
        // Llamada a la función CloseForm.
        ObjCTriangle.CloseForm(this);
    }
}
```

La clase **frmTriangle** tiene como datos miembro un objeto de tipo **CTriangle** llamado **ObjTriangle**, el cual es el encargado de acceder a las funciones públicas de la clase **CTriangle** para realizar diferentes cálculos y operaciones.

En el evento **Load** del formulario, cuyo procedimiento se llama **frmTriangle_Load()**, se utiliza el objeto **ObjTriangle**, para llamar a la función pública **InitializeData()**, que se encarga de inicializar los datos y controles del formulario. En el evento **Click** del botón **btnCalculate**, cuyo procedimiento se llama **btnCalculate_Click()**, se utiliza el objeto **ObjTriangle**, para llamar a las funciones públicas: **ReadData()**, **CheckTriangle()**, **PerimeterRectangle()**, **AreaRectangle()**, **PrintData()** y **PlotShape()**. En el evento **Click** del botón **btnReset**, cuyo procedimiento se llama **btnReset_Click()**, se utiliza el objeto **ObjTriangle**, para llamar a la función pública **InitializeData()**. En el evento **Click** del botón **btnClose**, cuyo procedimiento se llama **btnClose_Click()**, se utiliza el objeto **ObjTriangle**, para llamar a la función pública **CloseForm()**, que permite cerrar el formulario.

E) Pruebas

Para correr la aplicación basta con presionar la **tecla F5** o el **botón Start** de ejecución de la barra de menús de **Visual Studio .NET**. La Figura 1.5.8 muestra un ejemplo de la corrida de este programa, pero para saber que los resultados del problema son correctos, se debe examinar los resultados del programa cuidadosamente para asegurarnos que tengan sentido. El ejemplo mostrado de la salida del programa proporciona una buena prueba de la solución porque es relativamente fácil calcular el área y el perímetro manualmente o utilizando una calculadora. Para los valores de los tres lados que corresponde a 19, 16 y 17, se puede observar la gráfica de la figura geométrica en la **Caja de Imagen** llamada **picCanvas**. El perímetro deberá ser dos veces el semi-perímetro, que también es un valor fácil de calcular manualmente o utilizando una calculadora. El área deberá ser igual a la fórmula de Herón, que se puede calcular utilizando una calculadora. Para verificar que el programa funciona adecuadamente, se deberá ingresar unos pocos valores de prueba correspondientes a los tres lados de un triángulo. No se necesita probar más que con unos pocos casos de prueba para verificar que este programa está correcto.

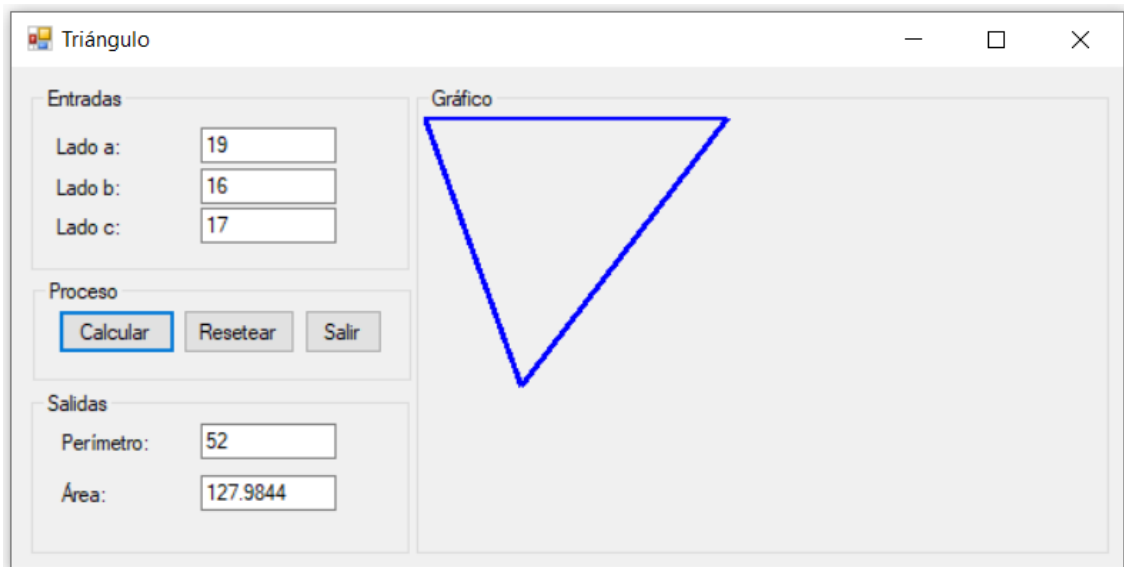


Figura 1.5.8. Ejemplo de la Corrida del Programa.

1.4. Praqniques Resueltas

PRAQNIQUE 1.1

Hexágono de 6 Colores

SINOPSIS	
Este pracnique explora la creación de un programa de gráficos por computadora para graficar un hexágono de 6 colores, dado el lado de un hexágono.	
Lenguaje:	C#
Compilador:	Visual Studio .NET
Habilidades:	Diseño de algoritmos, geometría computacional, clases y objetos, GUIs.
Nivel de experiencia:	Básico

A) Descripción del Problema

Dado el lado de un hexágono regular, dibujar la figura geométrica correspondiente, considerando que uno de los vértices del hexágono se encuentra sobre el eje de las 'x'. Además, se debe rellenar la figura con 6 colores, donde cada triángulo pequeño tiene un color como el color rojo, amarillo, verde, violeta, azul y celeste. (Ver Figura 1.1.1).

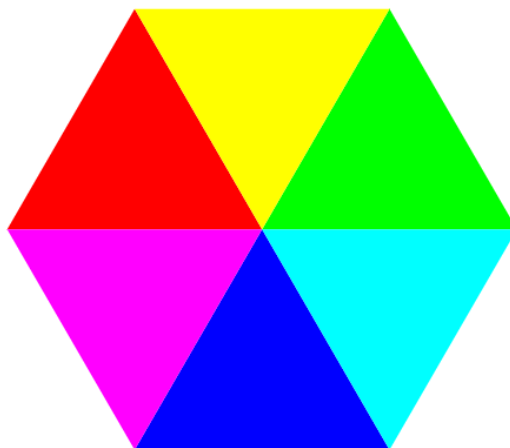


Figura 1.1.1. Hexágono de 6 colores.

B) Geometría de la Figura

Por construcción se encierra a la figura geométrica dentro de un rectángulo, luego se extienden algunos vértices hacia los lados del rectángulo formándose los segmentos 'b', 'Ap', que se calculan resolviendo el triángulo rectángulo APC. Estos dos segmentos permiten obtener los valores de los vértices del hexágono regular que se utilizarán para graficar la figura rellena con 6 colores. (Ver Figura 1.1.2).

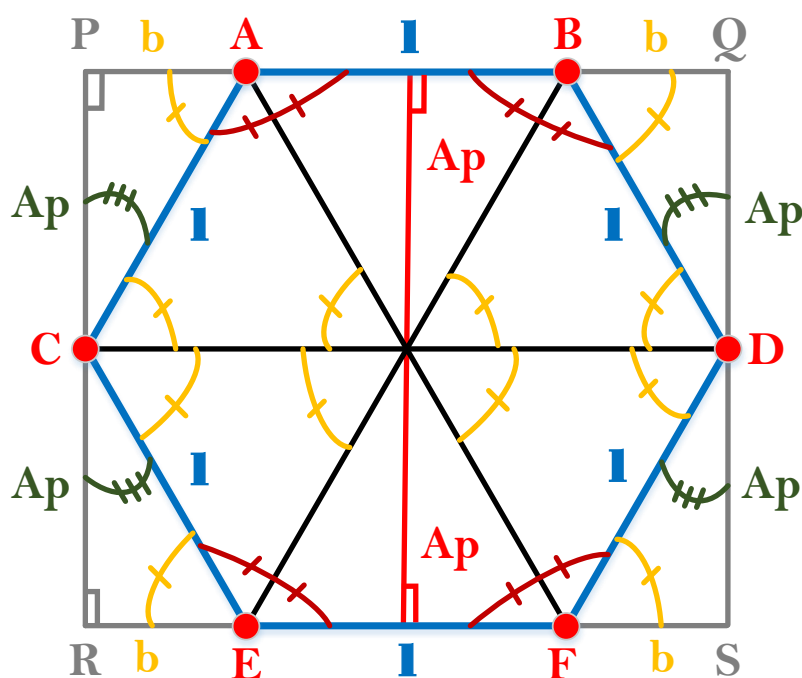


Figura 1.1.2. Vértices y ángulos del hexágono regular.

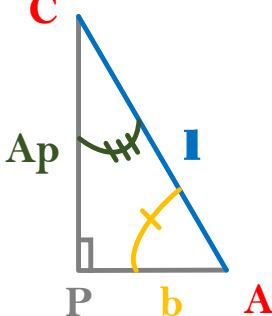
Ángulos

$$m\hat{1} = 60^\circ$$

$$m\hat{2} = 120^\circ$$

$$m\hat{3} = 30^\circ$$

Calculamos los valores del segmento '**b**' y de la apotema '**Ap**' en el triángulo rectángulo **APC**:

	$\cos(60) = \frac{b}{l}$ $b = l \cdot \cos(60)$	(1)
	$\text{sen}(60) = \frac{Ap}{l}$ $Ap = l \cdot \text{sen}(60)$	(2)

Luego, obtenemos los siguientes puntos del hexágono:

$$\begin{aligned} A(x_A, y_A) &= A(b, 0) \\ B(x_B, y_B) &= B(b + l, 0) \\ C(x_C, y_C) &= C(0, Ap) \\ D(x_D, y_D) &= D(2 \cdot b + l, Ap) \\ E(x_E, y_E) &= E(b, 2 \cdot Ap) \\ F(x_F, y_F) &= F(b + l, 2 \cdot Ap) \\ O(x_O, y_O) &= O(b + l/2, Ap) \end{aligned}$$

Con los valores del segmento '**b**' y de la 'apotema' se pueden calcular el valor del perímetro y del área del hexágono:

$$p = 6 \cdot l \quad (3)$$

$$\text{área} = \frac{p \cdot Ap}{2} \quad (4)$$

C) Algoritmos

1.1.1. Algoritmo de la Función *ReadData()*

1. Leer el lado del hexágono.

1.1.2. Algoritmo de la Función *CalculateVertex()*

1. Convertir el ángulo '**1**' de grados a radianes, multiplicando el valor del ángulo de 60 grados por PI radianes y dividiendo su resultado para 180 grados.
2. Calcular el valor del segmento '**b**' del contorno del hexágono.
3. Calcular las coordenadas del vértice '**A**'.
4. Calcular las coordenadas del vértice '**B**'.
5. Calcular las coordenadas del vértice '**C**'.
6. Calcular las coordenadas del vértice '**D**'.
7. Calcular las coordenadas del vértice '**E**'.
8. Calcular las coordenadas del vértice '**F**'.
9. Calcular las coordenadas del vértice central '**O**'.

1.1.5. Algoritmo de la Función *PlotShape()*

1. Asignar al objeto graph la funcionalidad de crear gráficos del PictureBox llamado picCanvas.

2. Llamada a la función CalculateVertex.
3. Actualizar el valor del vertice 'A' con lo que tiene el vértice 'A' multiplicado por un factor de escalamiento.
4. Actualizar el valor del vertice 'B' con lo que tiene el vértice 'B' multiplicado por un factor de escalamiento.
5. Actualizar el valor del vertice 'C' con lo que tiene el vértice 'C' multiplicado por un factor de escalamiento.
6. Actualizar el valor del vertice 'D' con lo que tiene el vértice 'D' multiplicado por un factor de escalamiento.
7. Actualizar el valor del vertice 'E' con lo que tiene el vértice 'E' multiplicado por un factor de escalamiento.
8. Actualizar el valor del vertice 'F' con lo que tiene el vértice 'F' multiplicado por un factor de escalamiento.
9. Actualizar el valor del vertice 'O' con lo que tiene el vértice 'O' multiplicado por un factor de escalamiento.
10. Crear una brocha sólida de color rojo (Red).
11. Crear un arreglo de puntos de tipo Point con los vértices: A, O, C, A.
12. Graficar un polígono relleno con la brocha y el arreglo de puntos del paso anterior.
13. Crear una brocha sólida de color amarillo (Yellow).
14. Crear un arreglo de puntos de tipo Point con los vértices: A, B, O, A.
15. Graficar un polígono relleno con la brocha y el arreglo de puntos del paso anterior.
16. Crear una brocha sólida de color verde (Green).
17. Crear un arreglo de puntos de tipo Point con los vértices: B, D, O, B.
18. Graficar un polígono relleno con la brocha y el arreglo de puntos del paso anterior.
19. Crear una brocha sólida de color azul violeta (BlueViolet).
20. Crear un arreglo de puntos de tipo Point con los vértices: C, O, E, C.
21. Graficar un polígono relleno con la brocha y el arreglo de puntos del paso anterior.
22. Crear una brocha sólida de color azul (Blue).
23. Crear un arreglo de puntos de tipo Point con los vértices: O, F, E, O.
24. Graficar un polígono relleno con la brocha y el arreglo de puntos del paso anterior.
25. Crear una brocha sólida de color celeste (SkyBlue).
26. Crear un arreglo de puntos de tipo Point con los vértices: O, D, F, O.
27. Graficar un polígono relleno con la brocha y el arreglo de puntos del paso anterior.

D) Código de la Aplicación

La implementación de la clase **CHexagon**, se presenta a continuación en la Tabla 1.1.2, que contiene datos miembro y funciones miembro que realizan diferentes operaciones.

Tabla 1.1.2. Código de la clase **CHexagon** del programa.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Drawing;
using System.Windows.Forms;

namespace WinAppHexagon
```